Projector-Generated Special Functions VIII

q-Hahn Lowest-Weight Packets, q-Racah Recoupling, Askey-Wilson Transfer Functions, and Deformation Geometry

Komorebi P.

Abstract

Papers VI and VII produced a continuous shape semigroup, positive discrete-series packet bases, classical Racah recoupling matrices, bistochastic measurement kernels, and pentagon coherence. The present paper introduces a positive deformation parameter q and rebuilds the same architecture inside a symmetric $U_q(\mathfrak{su}(1,1))$ coordinate system.

For $0 < q \le 1$, define the symmetric q-number

$$[x]_q = \frac{q^x - q^{-x}}{q - q^{-1}},$$

with the continuous value $[x]_q = x$ at $q = 1$. The positive shape packet $D^+_{\kappa,q}$ carries the ladder

$$K_0 \left|n\right\rangle = (n+\kappa)\left|n\right\rangle, \qquad K_+ \left|n\right\rangle = \sqrt{[n+1]_q\,[2\kappa+n]_q}\left|n+1\right\rangle,$$

$$K_- \left|n\right\rangle = \sqrt{[n]_q\,[2\kappa+n-1]_q}\left|n-1\right\rangle.$$

The coproduct

$$\Delta K_- = K_- \otimes q^{K_0} + q^{-K_0} \otimes K_-$$

produces exact fixed-excitation packet spaces and a q-Casimir. The lowest-weight vector of two packets has an explicit q-Hahn coefficient formula. Three packets generate two intermediate q-Casimir atlases whose overlap is orthogonal and belongs to the q-Racah class. Squaring the overlap produces a doubly stochastic measurement kernel.

For three equal half-shape packets at one excitation, the complete transform is

$$\boxed{\mathcal{R}_q = \begin{pmatrix} \dfrac{q}{1+q^2} & \dfrac{\sqrt{1+q^2+q^4}}{1+q^2} \\ \dfrac{\sqrt{1+q^2+q^4}}{1+q^2} & -\dfrac{q}{1+q^2} \end{pmatrix}.}$$

It converges to the classical Racah beam splitter at $q = 1$ and to a routing swap as $q \downarrow 0$. The corresponding row entropy and mixing gap are monotone in this two-channel case. A higher-excitation counterexample shows that such monotonicity is not universal.

Writing $q = \mathrm{e}^{-\theta}$ turns the recoupling matrices into a differentiable orthogonal flow. In a continuous sign gauge, the generator

$$A_N(\theta) = \mathcal{R}_N(\theta)^{\mathsf{T}} \partial_\theta \mathcal{R}_N(\theta)$$

is skew-symmetric. Its Frobenius norm defines a finite deformation metric. For four packets, five q-Casimir tree bases form the associahedral pentagon, and their ordered overlap product is the identity. Every finite q-Racah packet matrix can be compiled into Givens rotations and diagonal signs.

The scalar side is developed independently through the terminating q-Racah ${}_4\phi_3$ function and the continuous Askey-Wilson function. Exact duality, the $q \to 1$ Racah limit, and the Askey-Wilson q-difference equation are verified symbolically or at high precision. q-Racah and Askey-Wilson functions, quantum-group recoupling, and the pentagon identity are classical. The generated contribution is the common packet architecture that retains the q-Hahn seed, projector reduction, q-Racah analysis and

synthesis, stochastic measurement, deformation flow, associator certificate, and finite compiler in one system.

All figures were generated by the accompanying Python program. The release contains 32 vector figures, matching PNG files, 23 built-in checks, 20 independent tests, numerical tables, and the complete source.

Status and claim boundary

The q-Racah and Askey-Wilson polynomial families are classical members of the Askey-Wilson scheme. Their q-hypergeometric definitions, orthogonality measures, duality laws, q-difference equations, and classical limits are established parts of special-function theory. Their occurrence as Clebsch-Gordan or Racah coefficients in quantum-group representation theory is also classical.

This paper does not claim to have invented q-Racah polynomials, Askey-Wilson polynomials, quantum $6j$ symbols, or the abstract pentagon identity. It shows how they arise from the projector-resolved packet construction and adds a single operational language for lowest-weight reduction, analysis, synthesis, measurement, deformation transport, and compilation.

The finite matrix construction uses a symmetric q-number convention and the positive real range $0 < q \leq 1$. Other conventions are related by changes of generators and powers of q. Formula comparisons must therefore preserve the declared convention.

The q-dependent entropy statements concern the classical channel obtained after squaring the amplitudes and discarding phase. The underlying q-Racah matrix remains exactly reversible. Entropy is monotone under each fixed bistochastic channel. It is not generally monotone as the deformation parameter itself changes.

The four-body pentagon closes because every tree basis is complete in the same lowest-weight multiplicity space. The numerical product is a stringent implementation certificate, not a replacement for the abstract Biedenharn-Elliott theorem.

The final quantum discussion keeps this shape-recoupling module separate from the logarithmic number Hamiltonian used in the attached zeta-governed quantum dynamics. The modules can be tensored or composed, but the present paper does not identify their spectra and does not prove the Riemann hypothesis.

Contents

Status and claim boundary i

I q-deformed packet algebra 1

1 Symmetric q-numbers and positive shape packets 1

2 Coproduct, q-Casimir, and exact tensor routing 3

3 Fixed excitation and the lowest-weight multiplicity packet 4

II q-Hahn lowest-weight functions 5

4 Two-packet seed and closed coefficients 5

III q-Racah recoupling 7

5 Two intermediate q-Casimir atlases 7

6 An exact q-controlled beam splitter 10

7 Bistochastic measurement and entropy 12

IV Deformation geometry 16

8 Orthogonal transport in q 16

9 Classical and routing limits 18

V Four-body q-associators 19

10 Five complete q-Casimir tree atlases 19

11 Deforming the associator 21

VI Finite compilation 22

12 Givens factorization of a q-Racah packet 22

VII Scalar q-special functions 24

13 Terminating q-Racah function 24

14 Askey-Wilson transfer function 27

VIII Quantum and computational interpretation **29**

15 A reversible q-shape module **29**

16 What the deformation adds **30**

IX Verification and formulas **30**

17 Formula registry **30**

18 Numerical certificate table **31**

19 Research directions and boundaries **32**

20 Conclusion **32**

A Selected numerical tables **33**

B Complete Python implementation **33**

C Bibliographic note **53**

D Reproduction **54**

List of Figures

1.1 The q-deformed packet program. The positive-series ladder generates a q-Hahn lowest packet, pair q-Casimir atlases, q-Racah transforms, measurement kernels, deformation flow, and four-body associator closure. . . . 2
1.2 Symmetric q-numbers. As q approaches one, the deformed ladder approaches the ordinary linear coordinate. Smaller q amplifies large positive degrees. . . . 2
1.3 Positive-series raising coefficients. The local q-ladder remains exact while its high-degree growth changes strongly with the deformation parameter. . . . 3
2.1 Coproduct paths. A single lowering event is routed through each factor with the q-weights required by coassociativity. . . . 4
4.1 q-Hahn lowest-weight packets. The same total excitation is distributed differently as q changes. The alternating phase is omitted in the probability plot. . . . 6
4.2 Classical limit of the lowest packet. The q-Hahn coefficient vector converges to its ordinary Hahn counterpart as q approaches one. . . . 7
5.1 Pair q-Casimir spectra. The intermediate eigenvalues are exact q-number deformations of the classical quadratic spectrum. . . . 8
5.2 q-Racah recoupling matrices. Changing q redistributes amplitude across the same complete pair-Casimir coordinate systems. . . . 9
5.3 Analysis and synthesis residual. Across the tested dimensions and deformation values, the orthogonality error remains at floating-point scale. . . . 9
5.4 The opposing q-Casimir in one packet atlas. The tridiagonal form is the operator source of the q-Racah three-term recurrence. . . . 10
6.1 Closed beam-splitter amplitudes. The diagonal amplitude grows from zero to one half, while the transfer amplitude decreases from one to $\sqrt{3}/2$. . . . 11
6.2 Squared two-channel kernels. The coherent transform remains orthogonal, while the measured channel becomes a q-dependent bistochastic matrix. . . . 12
7.1 Bistochastic q-Racah kernels. Every amplitude matrix produces a classical measurement channel with unit row and column sums. . . . 13
7.2 Entropy and spectral gap in the exact two-channel model. Both increase monotonically with q. . . . 14
7.3 Higher-excitation entropy counterexample. A selected row of the q-Racah measurement channel is not monotone throughout the deformation interval. . . . 15
7.4 Spectral mixing beyond two channels. The q-dependence of the second eigenvalue becomes level dependent and need not follow a universal monotone law. . . . 15
8.1 Skew generator of the q-flow. The antisymmetric matrix records which packet coordinates exchange amplitude under an infinitesimal change of deformation. . . . 17
8.2 Deformation metric across excitation. The sensitivity of the complete recoupling atlas grows with multiplicity dimension. . . . 18
9.1 Classical limit of the matrix construction. After sign alignment, the complete q-Racah packet transform converges to the classical Racah transform. . . . 19
10.1 The q-deformed associahedral pentagon. Every vertex is a complete joint-Casimir atlas. Neighboring vertices are related by finite q-Racah packet transforms. . . . 20
10.2 Pentagon residual over q and excitation. The closure error remains at floating-point scale through the tested multiplicity dimensions. . . . 21
10.3 Five q-associator edges. Each edge has its own signed amplitude pattern, but their ordered product reconstructs the initial tree atlas. . . . 21
11.1 Deformation of the five associator edges. The q-flow redistributes local generator norm around the pentagon while preserving exact global closure. . . . 22
12.1 Rotation count. The direct compiler reaches the worst-case triangular count only when every subdiagonal elimination is needed. . . . 23
12.2 Compiler reconstruction residual. The finite rotation product reproduces the original q-Racah transform to numerical precision. . . . 23

12.3 Programmable q-packet execution path. The compiler begins with shapes, q, and total excitation, constructs only the required multiplicity projectors, and emits signed transforms, measurement kernels, or pentagon certificates. 24
13.1 A terminating q-Racah table. Degrees and lattice coordinates appear on equal footing, which makes duality visible before normalization by the orthogonality weights. 25
13.2 Duality residual. The parameter-exchanged degree-coordinate identity holds at floating-point precision. . . 26
13.3 q-Racah to Racah. The finite basic-hypergeometric polynomial converges termwise to its ordinary hypergeometric counterpart. 27
14.1 Askey-Wilson packet functions. Several degrees are plotted on the unit-circle coordinate $z = e^{i\theta}$. 28
14.2 q-difference certificate. Direct substitution of the generated functions into the Askey-Wilson operator leaves only numerical roundoff. 28
14.3 Finite and continuous q-functions. q-Hahn packets generate coupled bases, q-Racah functions generate finite recoupling, and Askey-Wilson functions provide the continuous spectral lift. 29

Part I

q-deformed packet algebra

Chapter 1

Symmetric q-numbers and positive shape packets

Definition 1.1 (Symmetric q-number)**.** For $q > 0$ and $q \neq 1$, define

$$\boxed{[x]_q = \frac{q^x - q^{-x}}{q - q^{-1}}.}$$

At $q = 1$, define $[x]_q = x$ by continuity.

Writing $q = \mathrm{e}^{-\theta}$ gives

$$[x]_q = \frac{\sinh(x\theta)}{\sinh\theta}.$$

Hence $[-x]_q = -[x]_q$ and $[x]_q > 0$ for $x > 0$.

Definition 1.2 (Positive q-shape packet)**.** For $\kappa > 0$, let $D^+_{\kappa,q}$ have orthonormal basis $\{|n\rangle : n \geq 0\}$ with

$$K_0 |n\rangle = (n + \kappa) |n\rangle ,$$

$$K_+ |n\rangle = \sqrt{[n+1]_q [2\kappa + n]_q}\, |n+1\rangle ,$$

$$K_- |n\rangle = \sqrt{[n]_q [2\kappa + n - 1]_q}\, |n-1\rangle .$$

Proposition 1.3 (Local q-algebra)**.** *On every basis vector away from a finite truncation boundary,*

$$[K_0, K_\pm] = \pm K_\pm, \qquad [K_-, K_+] = [2K_0]_q .$$

Proof. The first identities follow because $K_\pm$ change n by one. For the second, the coefficient of $|n\rangle$ in $K_-K_+ |n\rangle$ is

$$[n+1]_q [2\kappa + n]_q ,$$

while the coefficient in $K_+K_- |n\rangle$ is

$$[n]_q [2\kappa + n - 1]_q .$$

The elementary hyperbolic identity

$$[x+1]_q [y]_q - [x]_q [y-1]_q = [x+y]_q$$

with $x = n$ and $y = 2\kappa + n$ gives $[2n + 2\kappa]_q = [2K_0]_q$. □

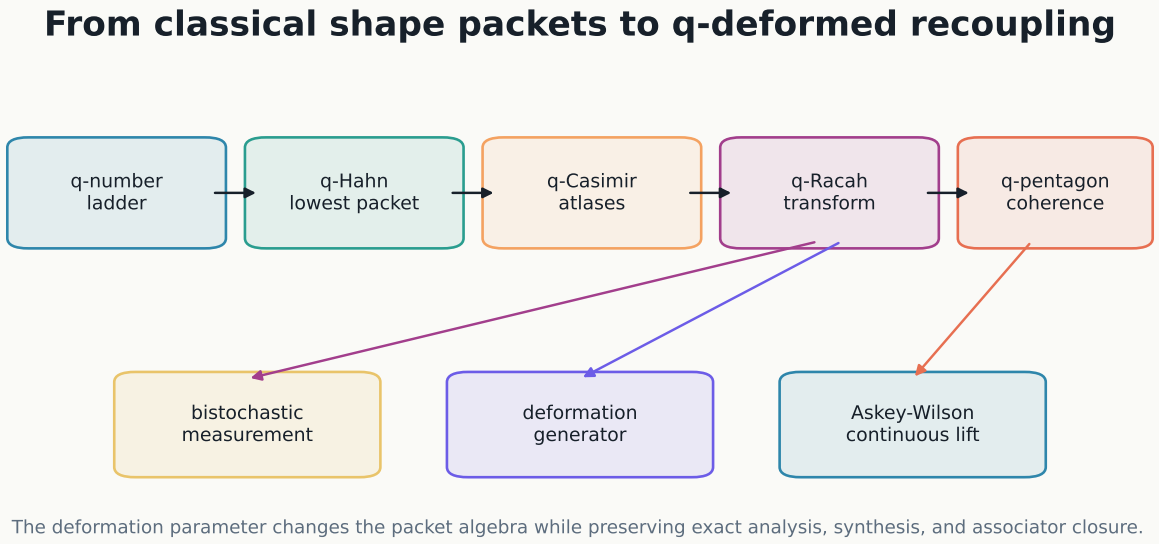


Figure 1.1: The q-deformed packet program. The positive-series ladder generates a q-Hahn lowest packet, pair q-Casimir atlases, q-Racah transforms, measurement kernels, deformation flow, and four-body associator closure.

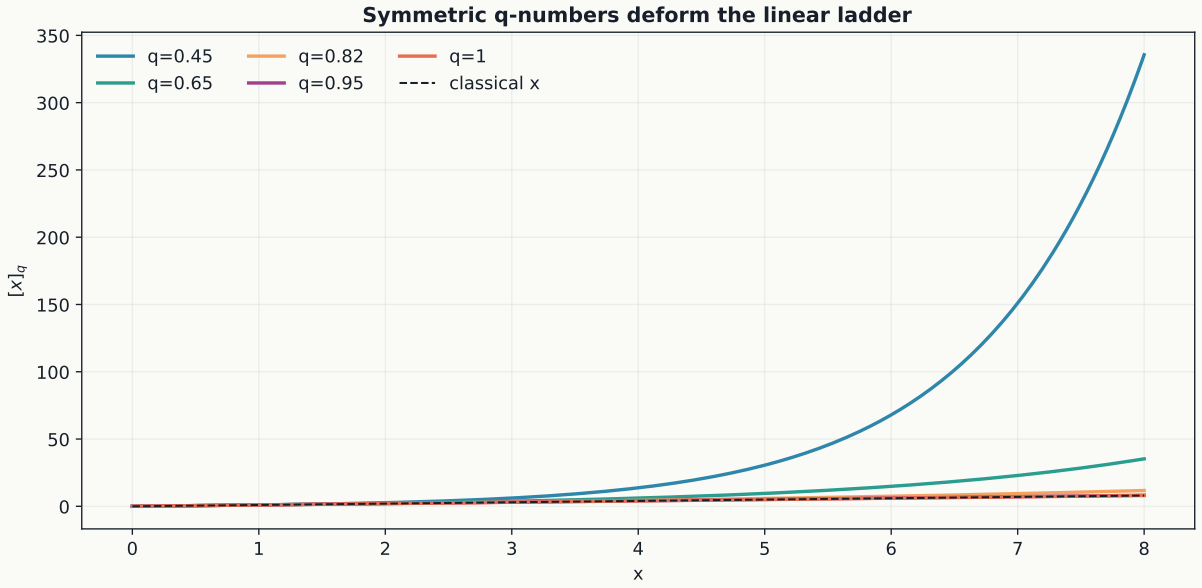


Figure 1.2: Symmetric q-numbers. As q approaches one, the deformed ladder approaches the ordinary linear coordinate. Smaller q amplifies large positive degrees.

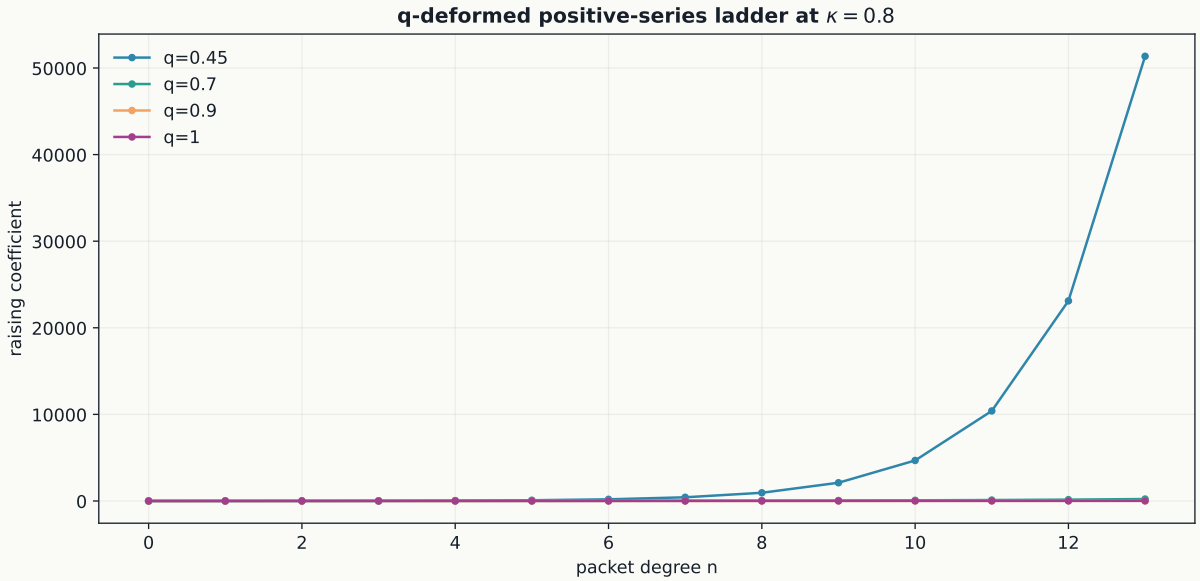


Figure 1.3: Positive-series raising coefficients. The local q-ladder remains exact while its high-degree growth changes strongly with the deformation parameter.

Chapter 2

Coproduct, q-Casimir, and exact tensor routing

Define

$$\Delta K_0 = K_0 \otimes \mathrm{I} + \mathrm{I} \otimes K_0,$$

$$\boxed{\Delta K_\pm = K_\pm \otimes q^{K_0} + q^{-K_0} \otimes K_\pm.}$$

Iterating this formula gives the total generators on any finite packet product.

Theorem 2.1 (Coassociative packet routing)**.** *The coproduct is coassociative on the declared generators. In particular,*

$$(\Delta \otimes \mathrm{I})\Delta K_- = (\mathrm{I} \otimes \Delta)\Delta K_-$$

and both sides equal

$$K_- \otimes q^{K_0} \otimes q^{K_0} + q^{-K_0} \otimes K_- \otimes q^{K_0} + q^{-K_0} \otimes q^{-K_0} \otimes K_-.$$

Proof. Since K_0 is primitive,

$$q^{\Delta K_0} = q^{K_0} \otimes q^{K_0}.$$

Substituting this identity into either iterated coproduct gives the same three terms. The proof for K_+ is identical. □

Definition 2.2 (q-Casimir)**.** Define

$$\boxed{\mathcal{C}_q = [K_0]_q\,[K_0 - 1]_q - K_+K_-.}$$

Theorem 2.3 (Scalar packet Casimir)**.** *On* $D^+_{\kappa,q}$*,*

$$\boxed{\mathcal{C}_q = [\kappa]_q\,[\kappa - 1]_q\,\mathrm{I}.}$$

Proof. On $|n\rangle$ the first term is

$$[n+\kappa]_q\,[n+\kappa-1]_q\,,$$

and the second is

$$[n]_q\,[2\kappa+n-1]_q\,.$$

The q-number identity

$$[x+y]_q\,[x+y-1]_q - [y]_q\,[2x+y-1]_q = [x]_q\,[x-1]_q$$

with $x=\kappa$ and $y=n$ gives the result. □

The symmetric coproduct routes one lowering event through every factor

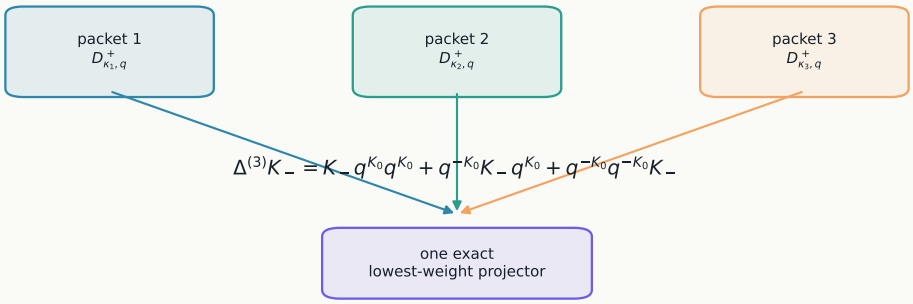


Figure 2.1: Coproduct paths. A single lowering event is routed through each factor with the q-weights required by coassociativity.

Chapter 3

Fixed excitation and the lowest-weight multiplicity packet

For m factors, let

$$\mathcal{H}_N^{(m)} = \operatorname{span}\{|n_1,\dots,n_m\rangle : n_1+\cdots+n_m = N\}.$$

Then

$$\dim \mathcal{H}_N^{(m)} = \binom{N+m-1}{m-1}.$$

The lowest-weight multiplicity packet is

$$\boxed{\mathcal{M}_N^{(m)} = \ker\left(K_-^{[m]}\big|_{\mathcal{H}_N^{(m)}}\right).}$$

Theorem 3.1 (Multiplicity dimension)**.** *For every $N \geq 0$ and $m \geq 2$,*

$$\boxed{\dim \mathcal{M}_N^{(m)} = \binom{N+m-2}{m-2}.}$$

Proof. The lowering map sends $\mathcal{H}_N^{(m)}$ to $\mathcal{H}_{N-1}^{(m)}$. Order the target compositions by their last coordinate. For every target state t, choose the source $t + e_m$. Lowering the last factor produces t with a nonzero coefficient. Lowering any earlier factor produces a target with a larger last coordinate. The corresponding square submatrix is triangular with nonzero diagonal, so the lowering map is onto. Rank-nullity gives

$$\dim \mathcal{M}_N^{(m)} = \binom{N+m-1}{m-1} - \binom{N+m-2}{m-1} = \binom{N+m-2}{m-2}.$$

□

This projector reduction is output-sensitive. The full tensor level defines the kernel. All recoupling then occurs in the smaller multiplicity packet.

Part II

q-Hahn lowest-weight functions

Chapter 4

Two-packet seed and closed coefficients

Take two shapes $\kappa_1, \kappa_2 > 0$ and total excitation N. Write the fixed-excitation basis as

$$|n, N-n\rangle, \qquad 0 \le n \le N.$$

Set

$$\lambda = q^{-(N-1+\kappa_1+\kappa_2)}.$$

Theorem 4.1 (Explicit q-Hahn lowest packet)**.** *A normalized lowest-weight vector is*

$$\boxed{|\Omega_N\rangle = \frac{1}{\sqrt{Z_N}} \sum_{n=0}^{N} (-\lambda)^n \sqrt{\begin{bmatrix} N \\ n \end{bmatrix}_q \frac{[2\kappa_2 + N - n]_{q,n}}{[2\kappa_1]_{q,n}}}\, |n, N-n\rangle,}$$

where

$$\begin{bmatrix} N \\ n \end{bmatrix}_q = \frac{[N]_q!}{[n]_q!\,[N-n]_q!}$$

and

$$[x]_{q,n} = \prod_{j=0}^{n-1} [x+j]_q.$$

Proof. Let c_n be the coefficient of $|n, N-n\rangle$. The coefficient of $|n, N-n-1\rangle$ in the total lowering equation is

$$q^{N-n-1+\kappa_2} \sqrt{[n+1]_q\,[2\kappa_1+n]_q}\, c_{n+1} + q^{-(n+\kappa_1)} \sqrt{[N-n]_q\,[2\kappa_2+N-n-1]_q}\, c_n.$$

Setting this to zero gives

$$\frac{c_{n+1}}{c_n} = -\lambda \sqrt{\frac{[N-n]_q\,[2\kappa_2+N-n-1]_q}{[n+1]_q\,[2\kappa_1+n]_q}}.$$

The displayed closed form has exactly this ratio. Normalization fixes Z_N. □

The squared coefficients form the discrete weight for the lowest q-Hahn packet. General coupled states are obtained by applying the total raising operator and are described by q-Hahn polynomial coordinates in the standard representation-theoretic normalization.

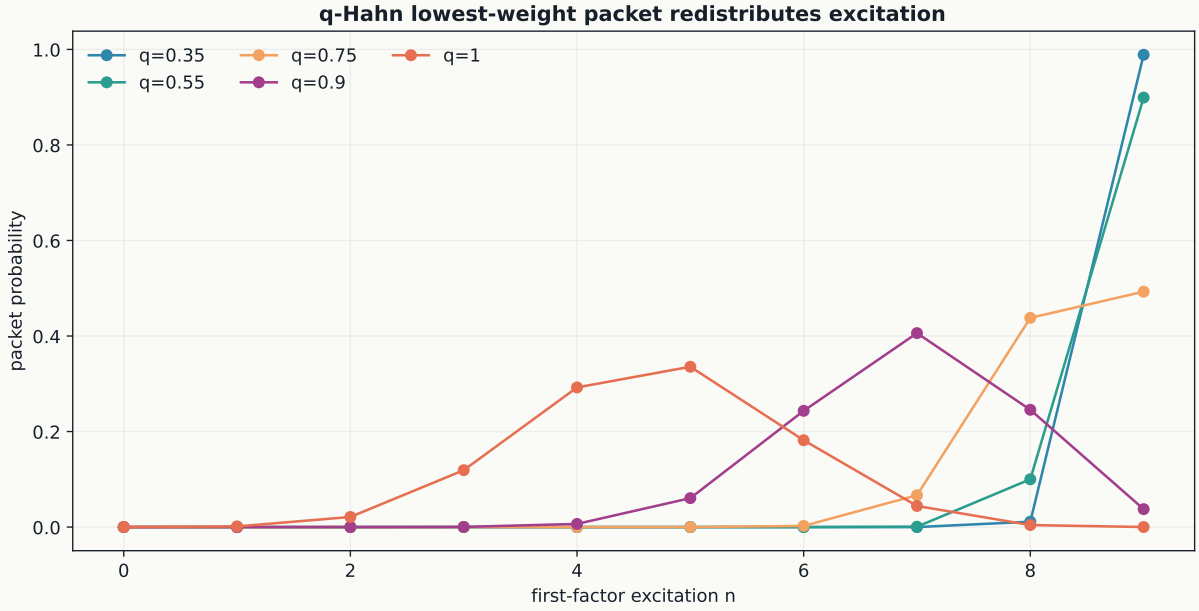


Figure 4.1: q-Hahn lowest-weight packets. The same total excitation is distributed differently as q changes. The alternating phase is omitted in the probability plot.

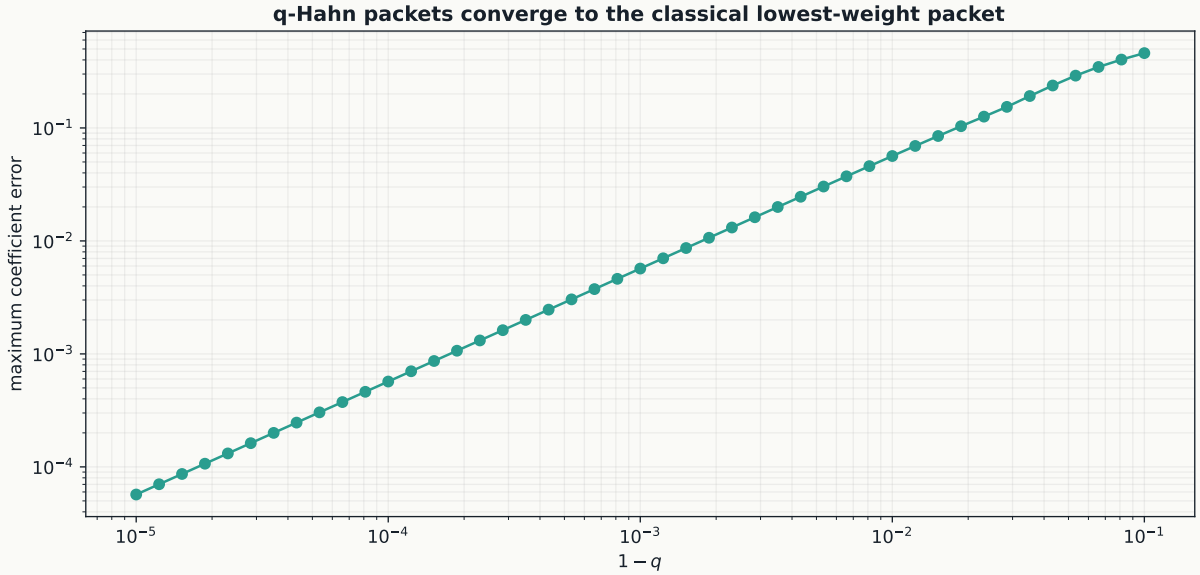


Figure 4.2: Classical limit of the lowest packet. The q-Hahn coefficient vector converges to its ordinary Hahn counterpart as q approaches one.

Worked example 4.2 (A five-excitation seed)**.** For

$$N = 5, \qquad (\kappa_1, \kappa_2) = (0.7, 1.1), \qquad q = 0.73,$$

the code constructs the six coefficients from the theorem and verifies

$$\left\| K_-^{[2]} \Omega_5 \right\|_\infty = 2.22 \times 10^{-16}.$$

The numerical residual is not the proof. It confirms that the implementation follows the exact recurrence.

Part III

q-Racah recoupling

Chapter 5

Two intermediate q-Casimir atlases

For three shapes, the multiplicity space has dimension $N + 1$. Restrict the pair Casimirs to this space:

$$\mathcal{C}_{12}^{(N)} = Q_N^{\mathsf{T}} \mathcal{C}_{12} Q_N, \qquad \mathcal{C}_{23}^{(N)} = Q_N^{\mathsf{T}} \mathcal{C}_{23} Q_N,$$

where the columns of Q_N form an orthonormal frame for $\mathcal{M}_N^{(3)}$.

The pair spectra are

$$\boxed{\lambda_j^{ab}(q) = [\kappa_a + \kappa_b + j]_q \, [\kappa_a + \kappa_b + j - 1]_q \,, \qquad 0 \le j \le N.}$$

Let L_N diagonalize $\mathcal{C}_{12}^{(N)}$ and let R_N diagonalize $\mathcal{C}_{23}^{(N)}$. The recoupling matrix is

$$\boxed{\mathcal{R}_N(q) = L_N^{\mathsf{T}} R_N.}$$

Theorem 5.1 (Exact analysis and synthesis)**.** *For every admissible finite packet,*

$$\boxed{\mathcal{R}_N(q)^{\mathsf{T}}\mathcal{R}_N(q) = \mathcal{R}_N(q)\mathcal{R}_N(q)^{\mathsf{T}} = \mathrm{I}_{N+1}.}$$

Proof. Both L_N and R_N are orthogonal eigenbases of real symmetric operators in the same multiplicity space. Therefore

$$\mathcal{R}_N^{\mathsf{T}}\mathcal{R}_N = R_N^{\mathsf{T}} L_N L_N^{\mathsf{T}} R_N = \mathrm{I}.$$

The other identity is identical. □

The classical representation-theoretic identification says that, after normalization and a parameter map determined by the three shapes and N, the entries of $\mathcal{R}_N$ are q-Racah functions. Equivalently, the restricted pair Casimirs form a finite tridiagonal pair. In the left atlas,

$$\mathcal{R}_N \operatorname{diag}(\lambda_m^{23})\mathcal{R}_N^{\mathsf{T}}$$

is tridiagonal. Hence each column obeys a three-term recurrence in the left packet label.

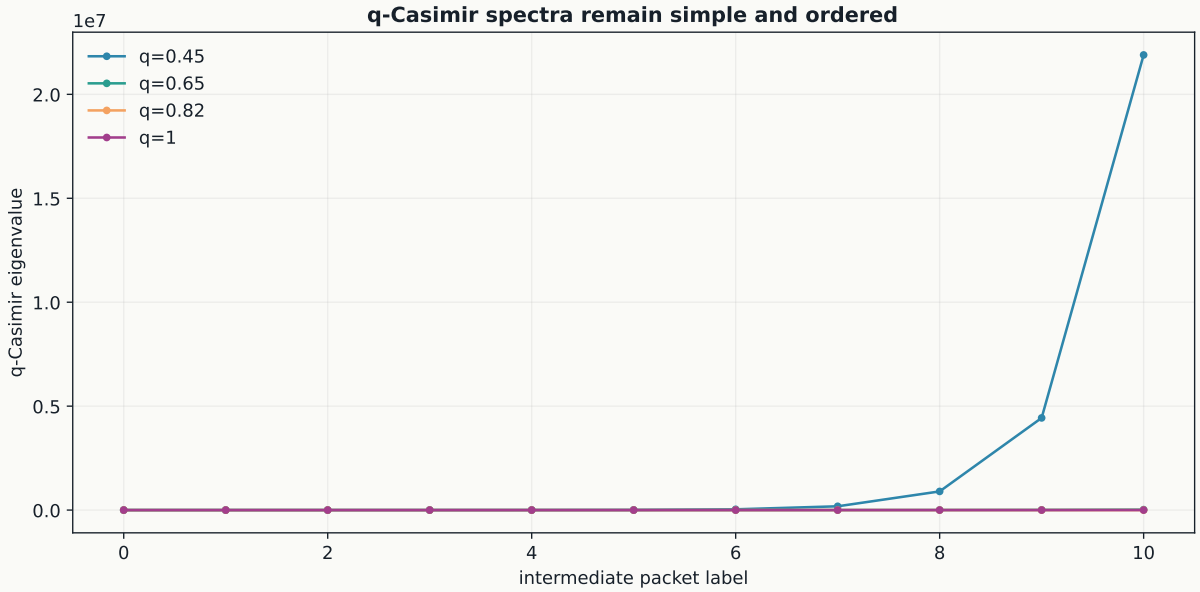


Figure 5.1: Pair q-Casimir spectra. The intermediate eigenvalues are exact q-number deformations of the classical quadratic spectrum.

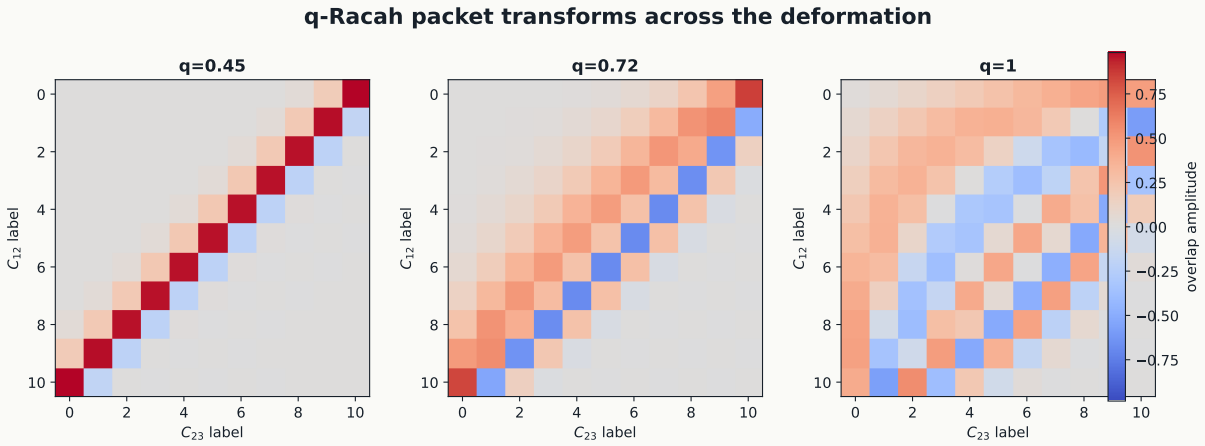


Figure 5.2: q-Racah recoupling matrices. Changing q redistributes amplitude across the same complete pair-Casimir coordinate systems.

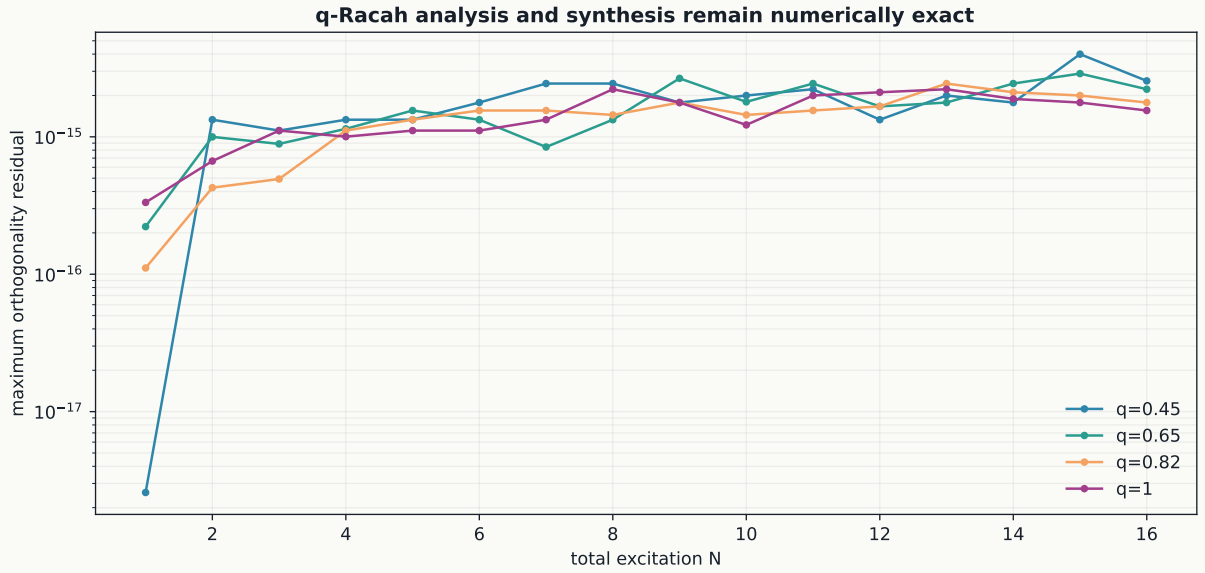


Figure 5.3: Analysis and synthesis residual. Across the tested dimensions and deformation values, the orthogonality error remains at floating-point scale.

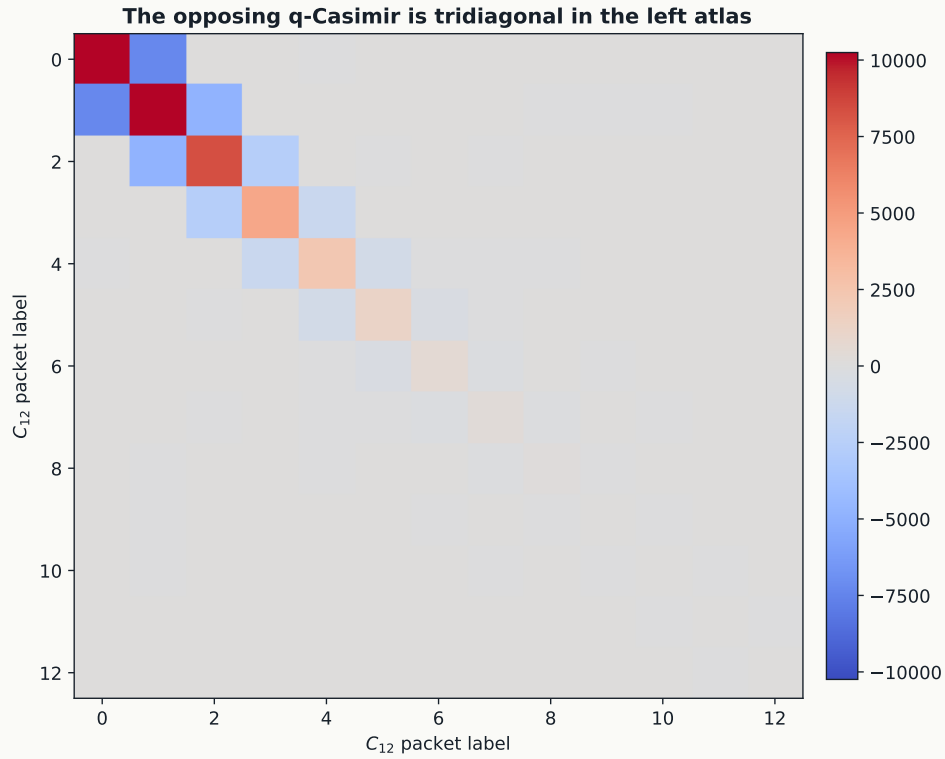


Figure 5.4: The opposing q-Casimir in one packet atlas. The tridiagonal form is the operator source of the q-Racah three-term recurrence.

Chapter 6

An exact q-controlled beam splitter

Take

$$\kappa_1 = \kappa_2 = \kappa_3 = \frac{1}{2}, \qquad N = 1.$$

Theorem 6.1 (Closed two-channel q-Racah transform)**.** *The left-to-right recoupling matrix can be chosen as*

$$\boxed{\mathcal{R}_q = \begin{pmatrix} c_q & s_q \\ s_q & -c_q \end{pmatrix},}$$

where

$$\boxed{c_q = \frac{q}{1+q^2}}$$

and

$$\boxed{s_q = \frac{\sqrt{1+q^2+q^4}}{1+q^2}.}$$

Proof. The fixed-excitation space is spanned by $|100\rangle, |010\rangle, |001\rangle$. The total lowering equation is

$$qx_1 + x_2 + q^{-1}x_3 = 0,$$

so the lowest-weight multiplicity space is two-dimensional. Restricting either pair Casimir to this plane gives two normalized eigenvectors. Their overlap simplifies to the displayed matrix. Directly,

$$c_q^2 + s_q^2 = \frac{q^2 + 1 + q^2 + q^4}{(1+q^2)^2} = 1,$$

so the matrix is orthogonal. Substitution into the restricted pair-Casimir eigenvalue equations completes the verification. □

At $q = 1$,

$$\mathcal{R}_1 = \begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ \frac{\sqrt{3}}{2} & -\frac{1}{2} \end{pmatrix},$$

which is the classical equal-shape Racah beam splitter. As $q \downarrow 0$,

$$\mathcal{R}_q \longrightarrow \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

The deformation therefore interpolates continuously between classical mixing and sparse routing.

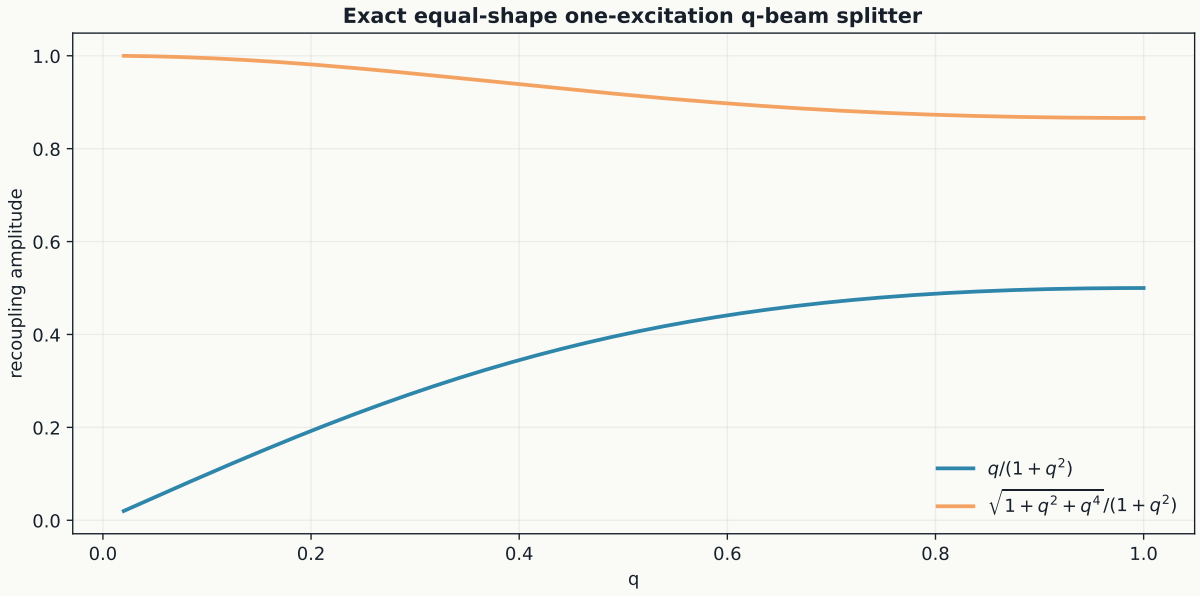


Figure 6.1: Closed beam-splitter amplitudes. The diagonal amplitude grows from zero to one half, while the transfer amplitude decreases from one to $\sqrt{3}/2$.

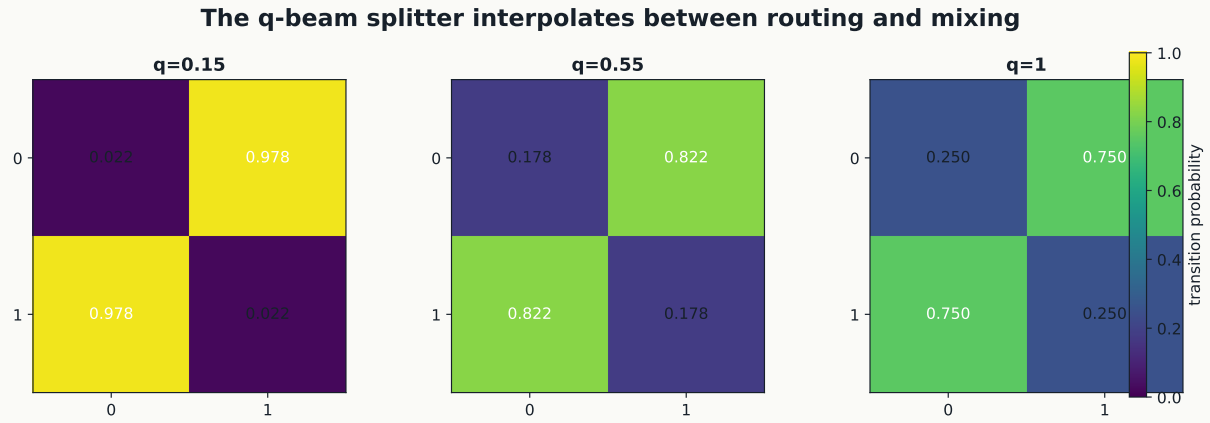


Figure 6.2: Squared two-channel kernels. The coherent transform remains orthogonal, while the measured channel becomes a q-dependent bistochastic matrix.

Chapter 7

Bistochastic measurement and entropy

Define

$$\boxed{P_{jm}(q) = |\mathcal{R}_N(q)_{jm}|^2.}$$

Theorem 7.1 (Bistochastic q-Racah channel)**.** *For every finite recoupling matrix,*

$$P_{jm} \geq 0, \qquad \sum_m P_{jm} = 1, \qquad \sum_j P_{jm} = 1.$$

Consequently, for every probability row vector p,

$$\boxed{\mathrm{H}(pP) \geq \mathrm{H}(p),}$$

where H *is Shannon entropy.*

Proof. The row and column sums are the squared norms of rows and columns of the orthogonal matrix. Every doubly stochastic matrix is a convex combination of permutation matrices. Shannon entropy is concave and invariant under permutations, so it cannot decrease. □

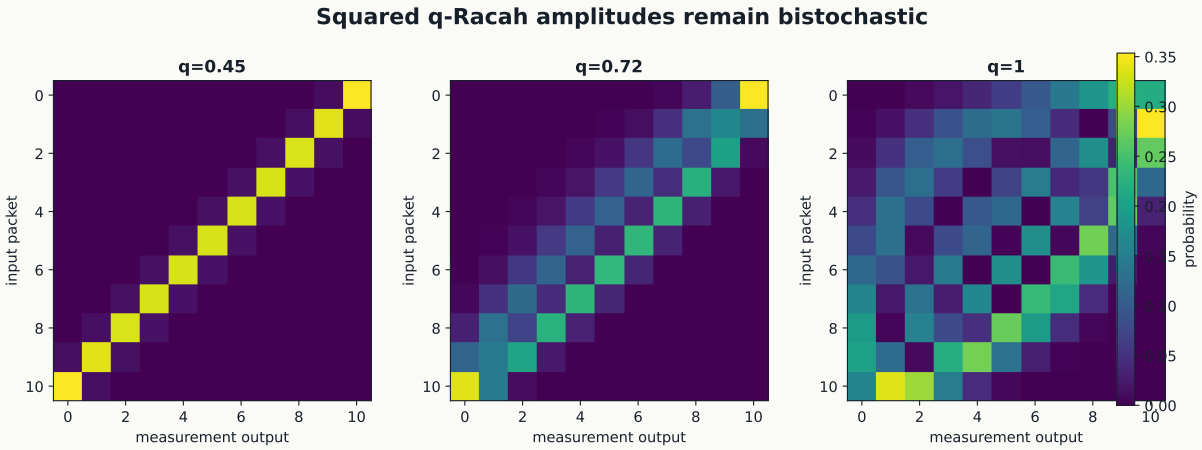


Figure 7.1: Bistochastic q-Racah kernels. Every amplitude matrix produces a classical measurement channel with unit row and column sums.

Theorem 7.2 (Monotone one-excitation mixing)**.** *For the equal-shape one-excitation channel and* $0 < q \leq 1$*, the row entropy and the spectral mixing gap increase with* q*.*

Proof. The smaller row probability is

$$p(q) = c_q^2 = \frac{q^2}{(1+q^2)^2}.$$

It increases from zero to $1/4$. The binary entropy

$$h(p) = -p \log p - (1-p) \log(1-p)$$

is increasing for $0 \leq p \leq 1/2$. The two eigenvalues of the channel are 1 and $2p-1$, so the spectral gap is

$$\boxed{1 - |2p-1| = 2p = \frac{2q^2}{(1+q^2)^2},}$$

which is also increasing on $(0, 1]$. □

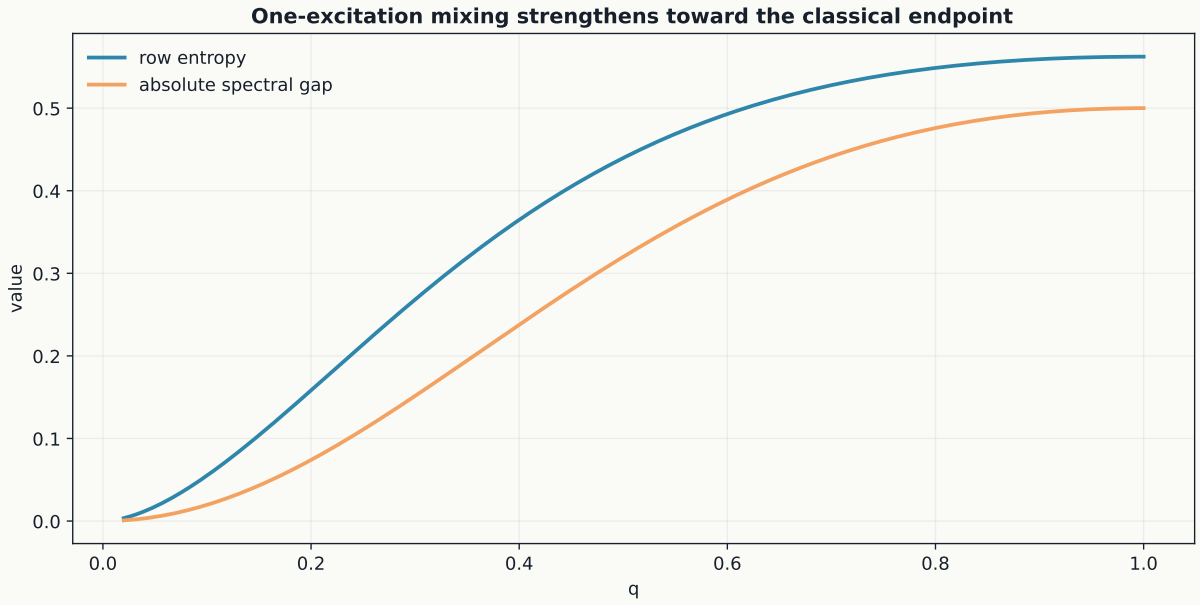


Figure 7.2: Entropy and spectral gap in the exact two-channel model. Both increase monotonically with q.

Boundary statement 7.3 (No universal entropy monotonicity in q). For higher excitation, each fixed q still gives a bistochastic channel and therefore increases entropy relative to its input. However, the row entropy need not be monotone as q varies. At $N = 3$ and equal half-shapes, the second row entropy decreases from approximately 1.24488 to 1.23710 between $q \approx 0.7772$ and $q \approx 0.7884$. This negative control prevents the one-excitation theorem from being promoted to all packet levels.

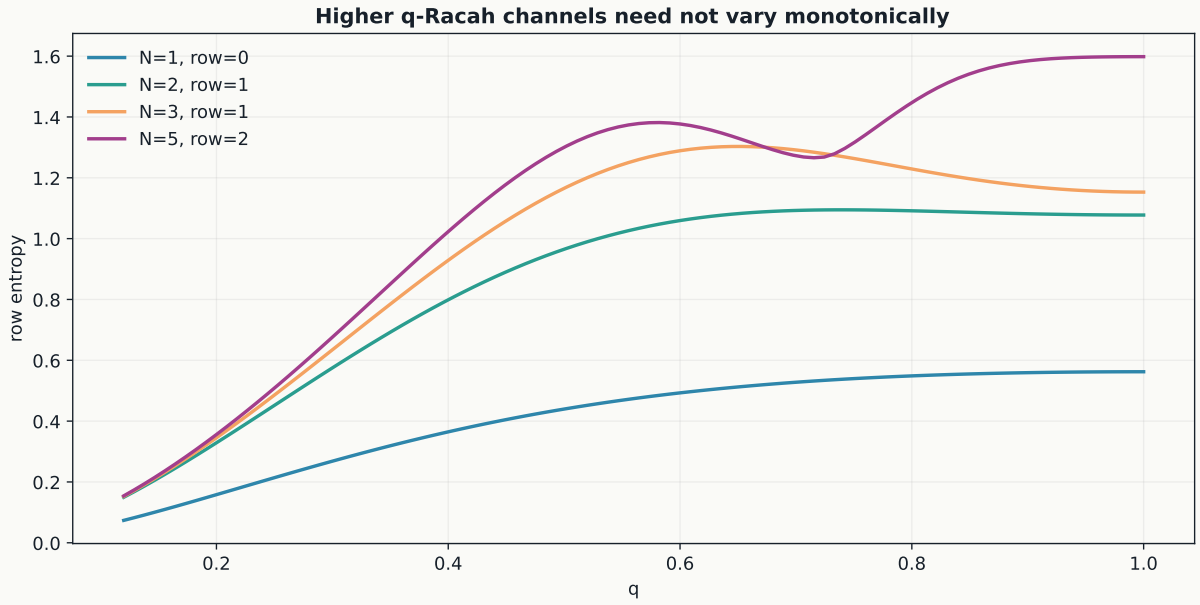


Figure 7.3: Higher-excitation entropy counterexample. A selected row of the q-Racah measurement channel is not monotone throughout the deformation interval.

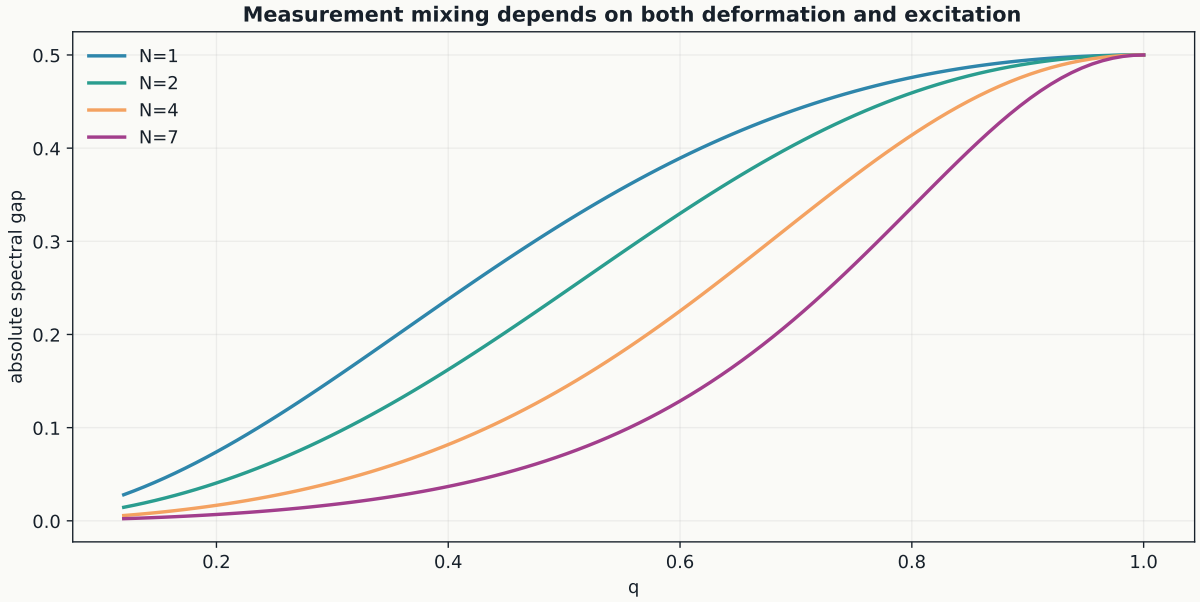


Figure 7.4: Spectral mixing beyond two channels. The q-dependence of the second eigenvalue becomes level dependent and need not follow a universal monotone law.

Part IV

Deformation geometry

Chapter 8

Orthogonal transport in q

Write

$$q = \mathrm{e}^{-\theta}, \qquad \theta \geq 0.$$

Choose a continuous sign gauge for the eigenvectors away from degeneracies and define

$$R_N(\theta) = \mathcal{R}_N(\mathrm{e}^{-\theta}).$$

Theorem 8.1 (Skew deformation generator)**.** *The matrix*

$$\boxed{A_N(\theta) = R_N(\theta)^{\mathsf{T}} \partial_\theta R_N(\theta)}$$

is skew-symmetric, and

$$\boxed{\partial_\theta R_N = R_N A_N.}$$

Proof. Differentiate

$$R_N^{\mathsf{T}} R_N = \mathrm{I}.$$

This gives

$$(\partial_\theta R_N)^{\mathsf{T}} R_N + R_N^{\mathsf{T}} \partial_\theta R_N = 0,$$

so $A_N^{\mathsf{T}} = -A_N$. Multiplying the definition of A_N on the left by R_N gives the tangent equation. □

Define the finite deformation metric

$$\boxed{g_N(\theta) = \frac{1}{2} \|A_N(\theta)\|_F^2 .}$$

It is invariant under a constant orthogonal change of either packet atlas. A q-dependent sign jump is not a smooth gauge and must be removed before differentiation.

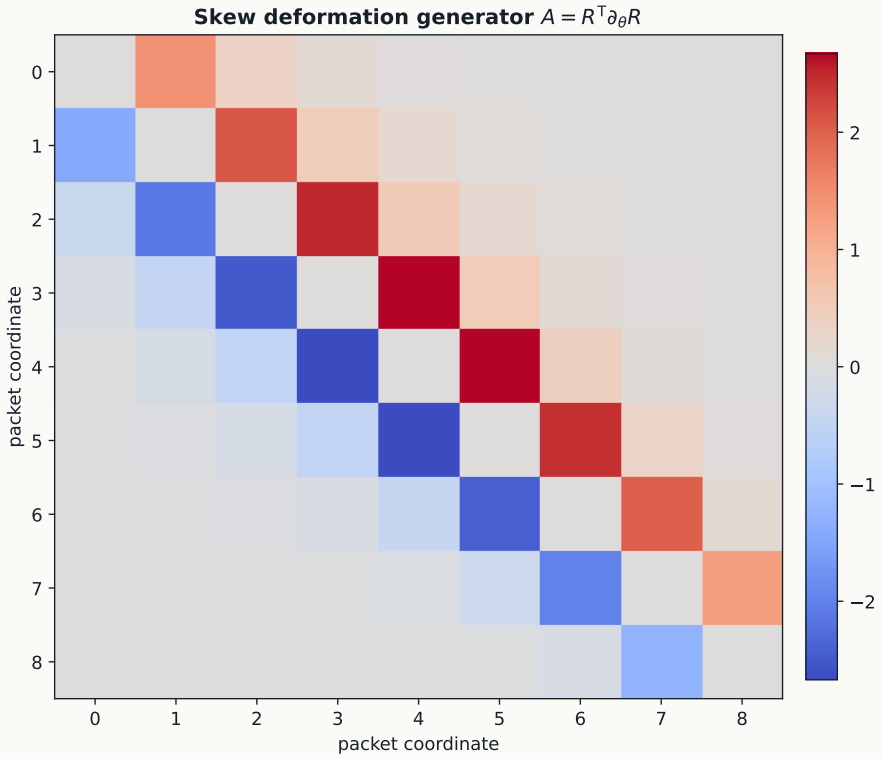


Figure 8.1: Skew generator of the q-flow. The antisymmetric matrix records which packet coordinates exchange amplitude under an infinitesimal change of deformation.

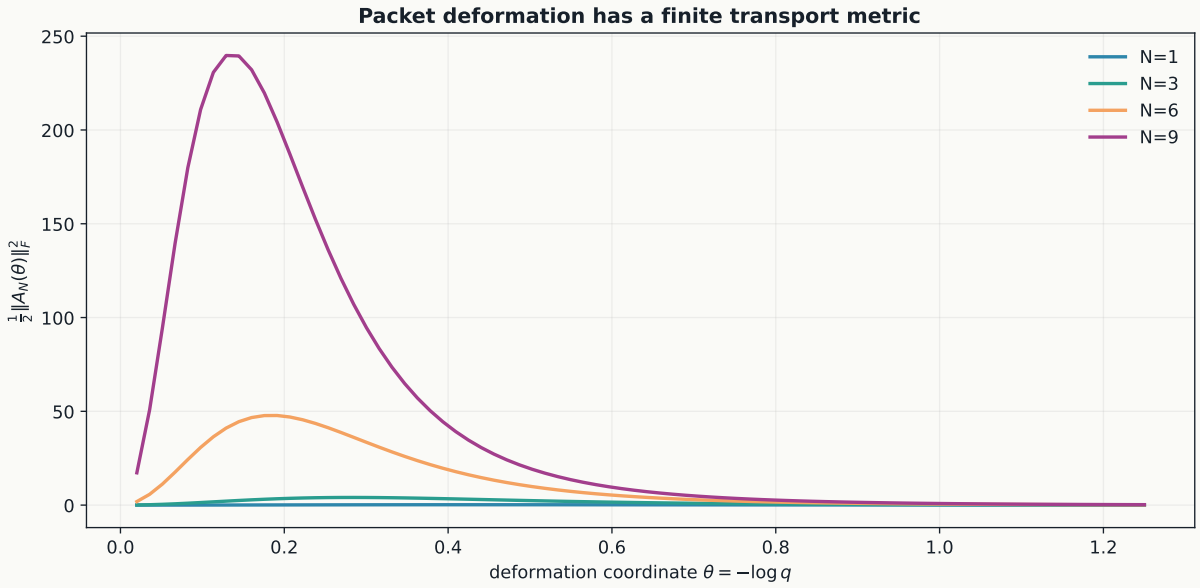


Figure 8.2: Deformation metric across excitation. The sensitivity of the complete recoupling atlas grows with multiplicity dimension.

Chapter 9

Classical and routing limits

The q-Hahn and q-Racah systems have two useful endpoints.

At $q \to 1$, symmetric q-numbers satisfy

$$[x]_q = x + O((\log q)^2),$$

so the packet algebra, Casimir spectra, lowest-weight vectors, and recoupling matrices converge to their ordinary $\mathfrak{su}(1,1)$ counterparts.

At $q \downarrow 0$, no uniform finite limit exists for every unrenormalized q-number expression. Nevertheless, selected normalized packet transforms have finite routing limits. The one-excitation beam splitter is the simplest exact example.

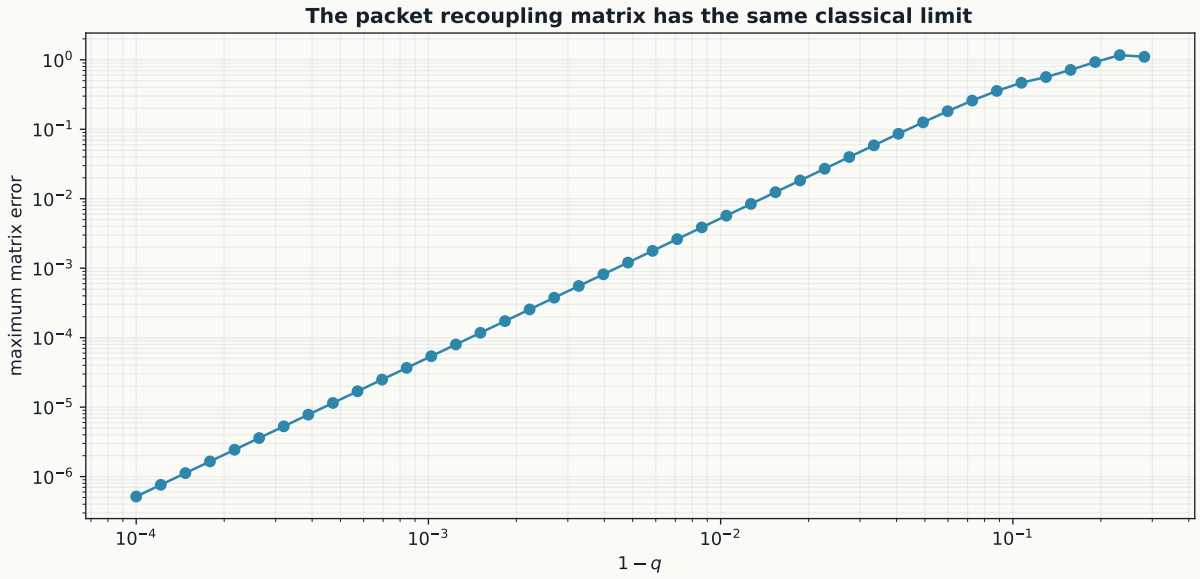


Figure 9.1: Classical limit of the matrix construction. After sign alignment, the complete q-Racah packet transform converges to the classical Racah transform.

Part V

Four-body q-associators

Chapter 10

Five complete q-Casimir tree atlases

For four packets, the multiplicity dimension is

$$\boxed{\dim \mathcal{M}_N^{(4)} = \binom{N+2}{2}.}$$

Consider the five binary coupling trees

$$(((12)3)4), \quad ((1(23))4), \quad (1((23)4)), \quad (1(2(34))), \quad ((12)(34)).$$

Each tree determines two commuting intermediate q-Casimirs. Their joint eigenvectors form a complete orthonormal basis B_r of the same multiplicity space.

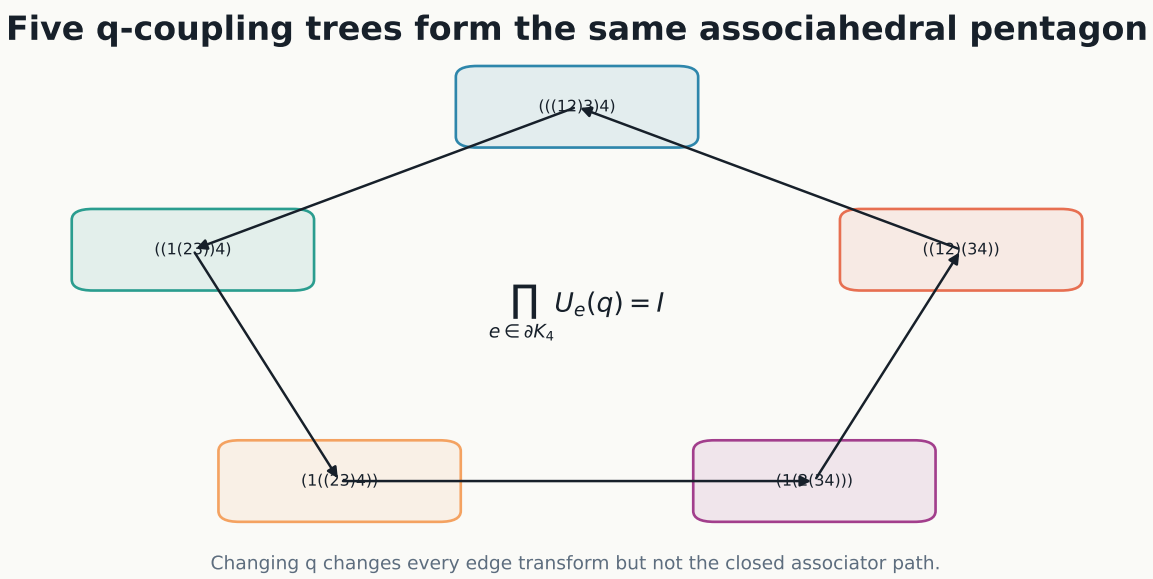


Figure 10.1: The q-deformed associahedral pentagon. Every vertex is a complete joint-Casimir atlas. Neighboring vertices are related by finite q-Racah packet transforms.

Theorem 10.1 (Finite q-pentagon certificate)**.** *Let*

$$U_{r,r+1} = B_r^{\mathsf{T}} B_{r+1},$$

with indices taken modulo five. Then

$$\boxed{U_{01}U_{12}U_{23}U_{34}U_{40} = \mathrm{I}.}$$

Proof. Insert the definitions and use completeness:

$$B_0^{\mathsf{T}} B_1 B_1^{\mathsf{T}} B_2 \cdots B_4^{\mathsf{T}} B_0 = B_0^{\mathsf{T}} B_0 = \mathrm{I}.$$

□

This linear-algebra proof explains why the numerical pentagon is a strong implementation test. Any wrong projector, packet order, omitted vector, or inconsistent tree basis prevents the telescoping product from closing.

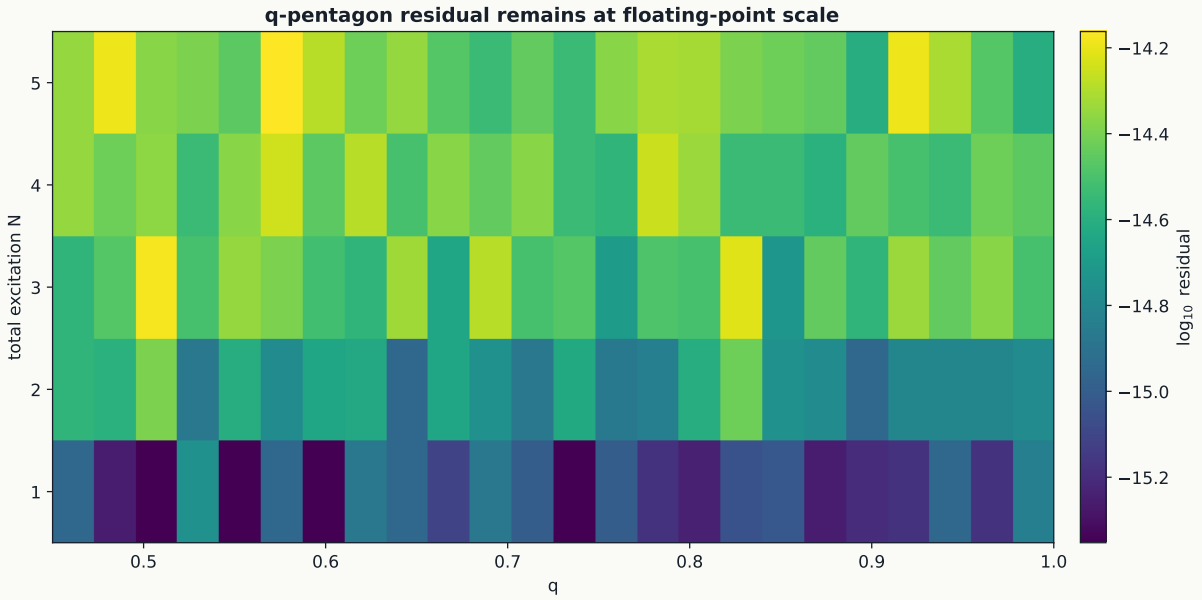


Figure 10.2: Pentagon residual over q and excitation. The closure error remains at floating-point scale through the tested multiplicity dimensions.

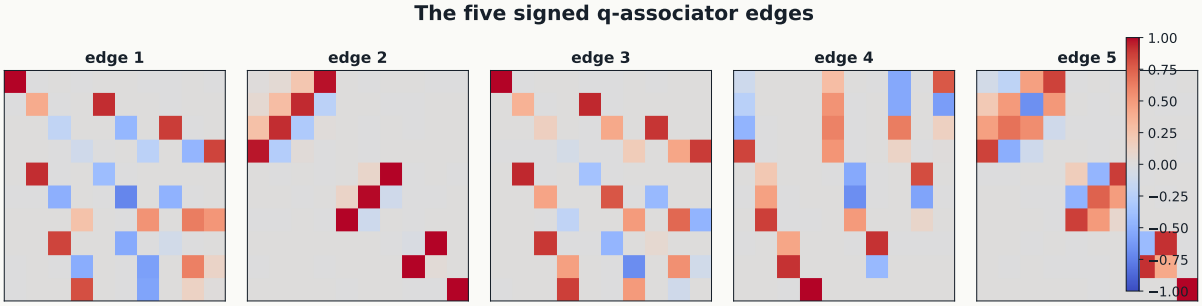


Figure 10.3: Five q-associator edges. Each edge has its own signed amplitude pattern, but their ordered product reconstructs the initial tree atlas.

Chapter 11

Deforming the associator

Each edge matrix depends on $\theta = -\log q$. In a continuous gauge, define

$$A_r(\theta) = U_r(\theta)^{\mathsf{T}} \partial_\theta U_r(\theta).$$

Every A_r is skew-symmetric. Differentiating the pentagon identity gives a transported sum of edge generators equal to zero. This is the finite flatness equation for the q-associator loop.

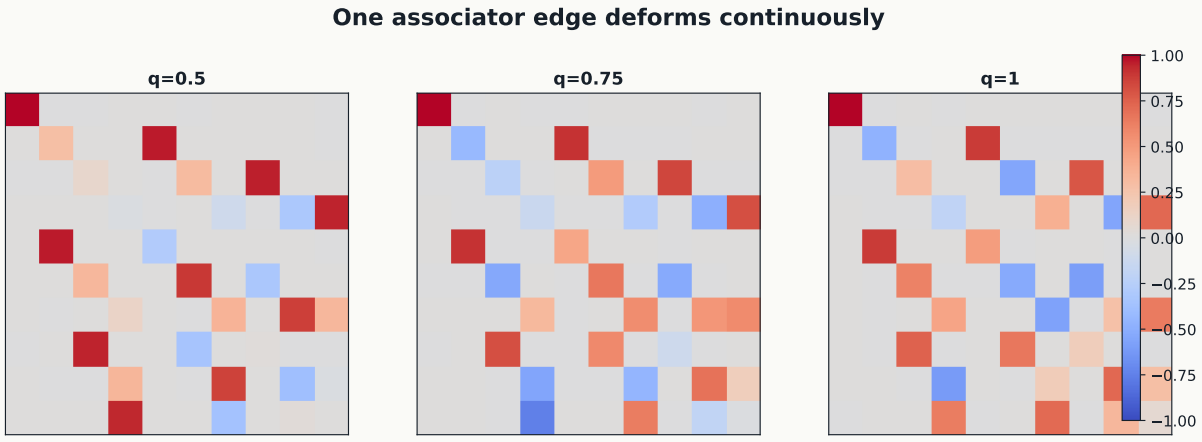


Figure 11.1: Deformation of the five associator edges. The q-flow redistributes local generator norm around the pentagon while preserving exact global closure.

Part VI

Finite compilation

Chapter 12

Givens factorization of a q-Racah packet

Theorem 12.1 (Finite rotation compiler)**.** *Every real orthogonal $d \times d$ q-Racah packet matrix admits a factorization*

$$\boxed{R = G_1^{\mathsf{T}} \cdots G_L^{\mathsf{T}} D,}$$

where D is diagonal with entries ± 1 and every G_j is a two-coordinate Givens rotation. One may choose

$$\boxed{L \leq \frac{d(d-1)}{2}.}$$

Proof. Apply row Givens rotations from the lower left upward to eliminate all entries below the diagonal. Orthogonality forces the resulting upper triangular matrix to be diagonal with entries ± 1. Reversing the rotations reconstructs R. □

The compiler acts in the multiplicity packet. It does not require a dense gate acting on the full tensor-product level.

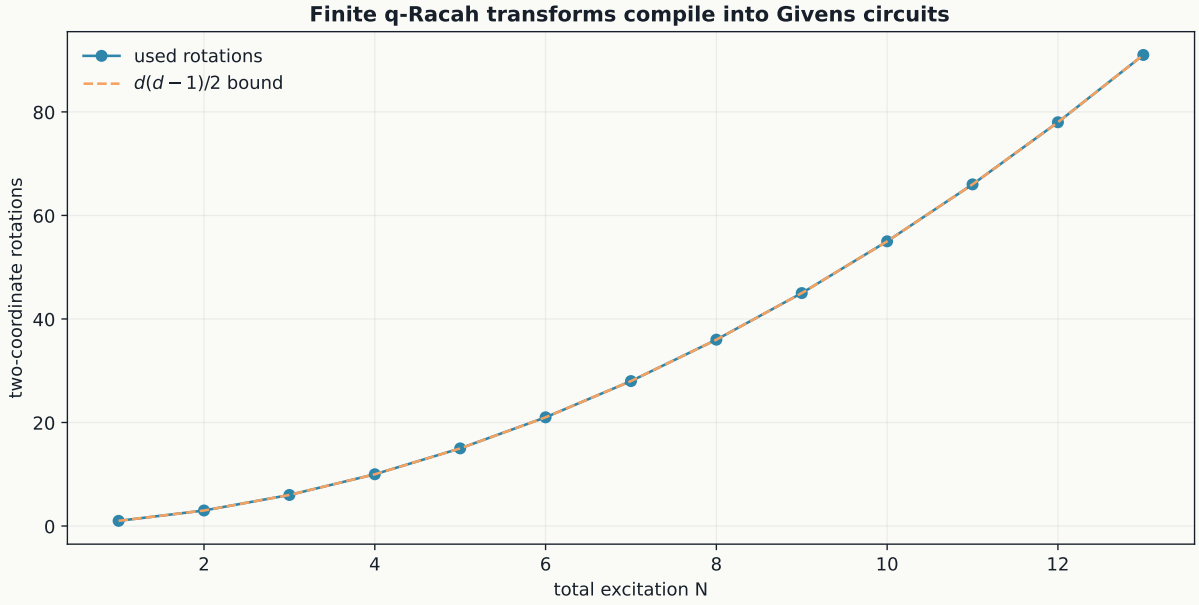


Figure 12.1: Rotation count. The direct compiler reaches the worst-case triangular count only when every subdiagonal elimination is needed.

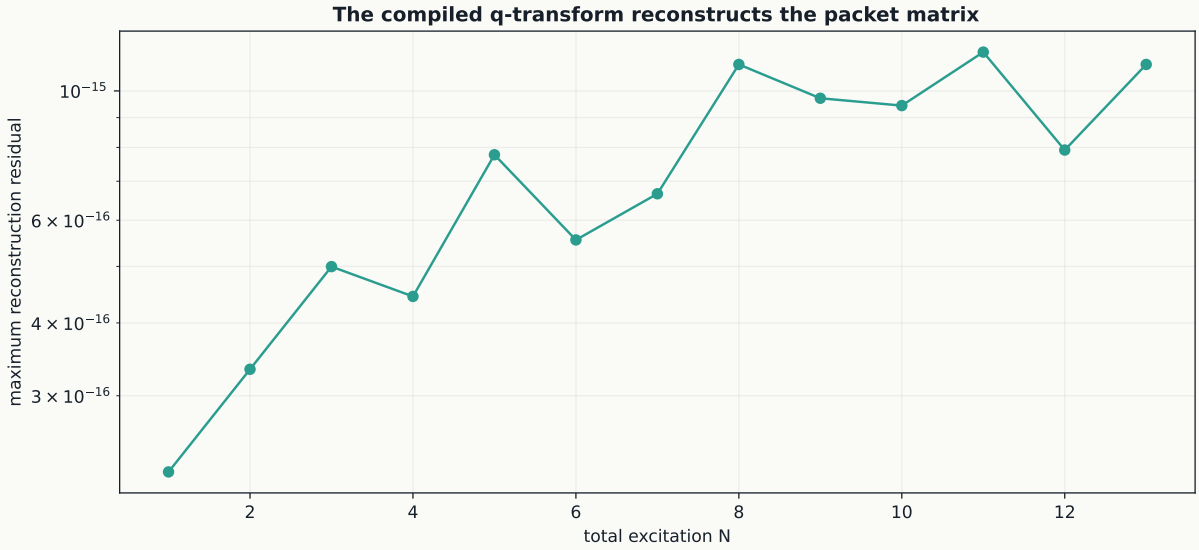


Figure 12.2: Compiler reconstruction residual. The finite rotation product reproduces the original q-Racah transform to numerical precision.

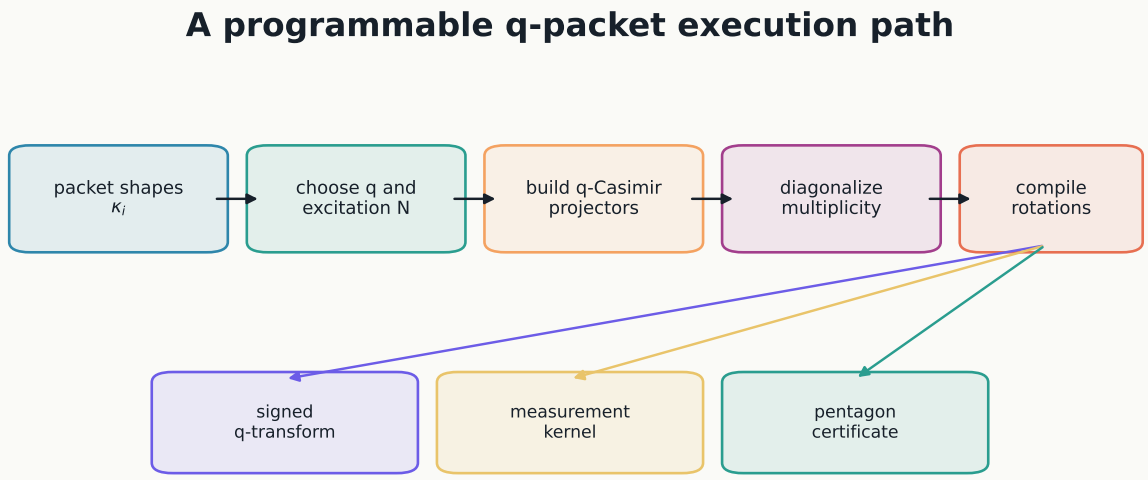


Figure 12.3: Programmable q-packet execution path. The compiler begins with shapes, q, and total excitation, constructs only the required multiplicity projectors, and emits signed transforms, measurement kernels, or pentagon certificates.

Part VII

Scalar q-special functions

Chapter 13

Terminating q-Racah function

For the q-Pochhammer symbol

$$(a;q)_n = \prod_{j=0}^{n-1}(1-aq^j),$$

define

$$\boxed{R_n(\mu(y);\alpha,\beta,\gamma,\delta \mid q) = {}_4\phi_3\left(\begin{matrix} q^{-n}, \alpha\beta q^{n+1}, q^{-y}, \gamma\delta q^{y+1} \\ \alpha q, \beta\delta q, \gamma q \end{matrix}; q, q\right),}$$

where

$$\mu(y) = q^{-y} + \gamma\delta q^{y+1}.$$

The series terminates at n. A finite orthogonal family arises when one denominator parameter is q^{-N}.

Theorem 13.1 (q-Racah duality)**.** *The terminating function satisfies*

$$\boxed{R_n(\mu(y);\alpha,\beta,\gamma,\delta \mid q) = R_y(\mu(n);\gamma,\delta,\alpha,\beta \mid q).}$$

Proof. After exchanging n with y and (α, β) with (γ, δ), the four numerator q-Pochhammer factors are merely permuted. The three denominator factors are also permuted because $\delta\beta q = \beta\delta q$. Every summand is unchanged. □

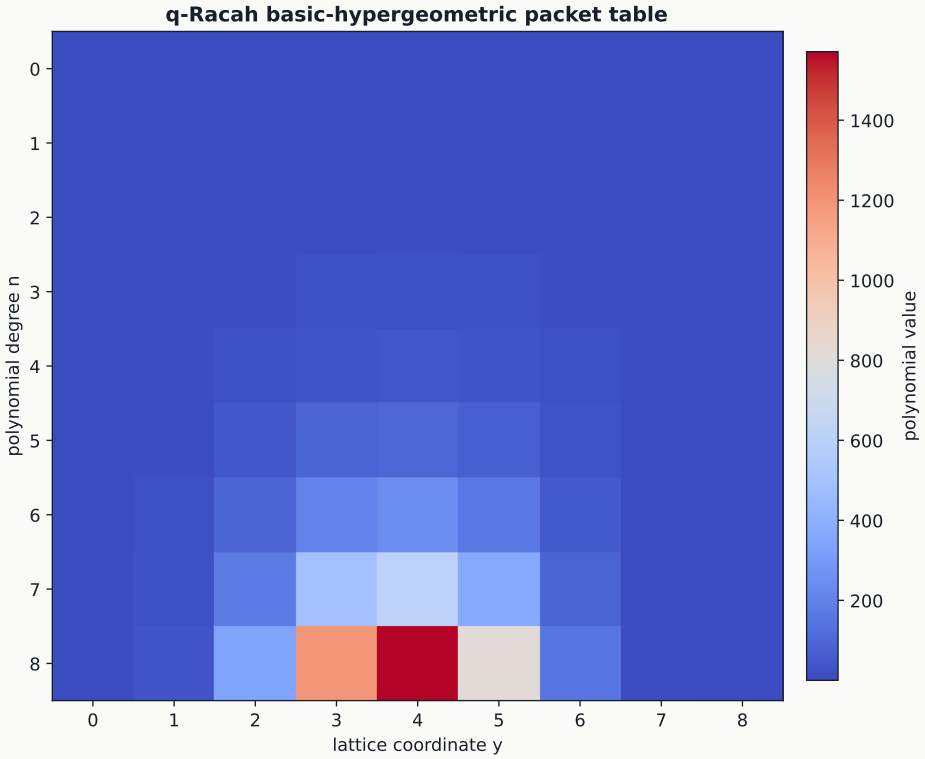


Figure 13.1: A terminating q-Racah table. Degrees and lattice coordinates appear on equal footing, which makes duality visible before normalization by the orthogonality weights.

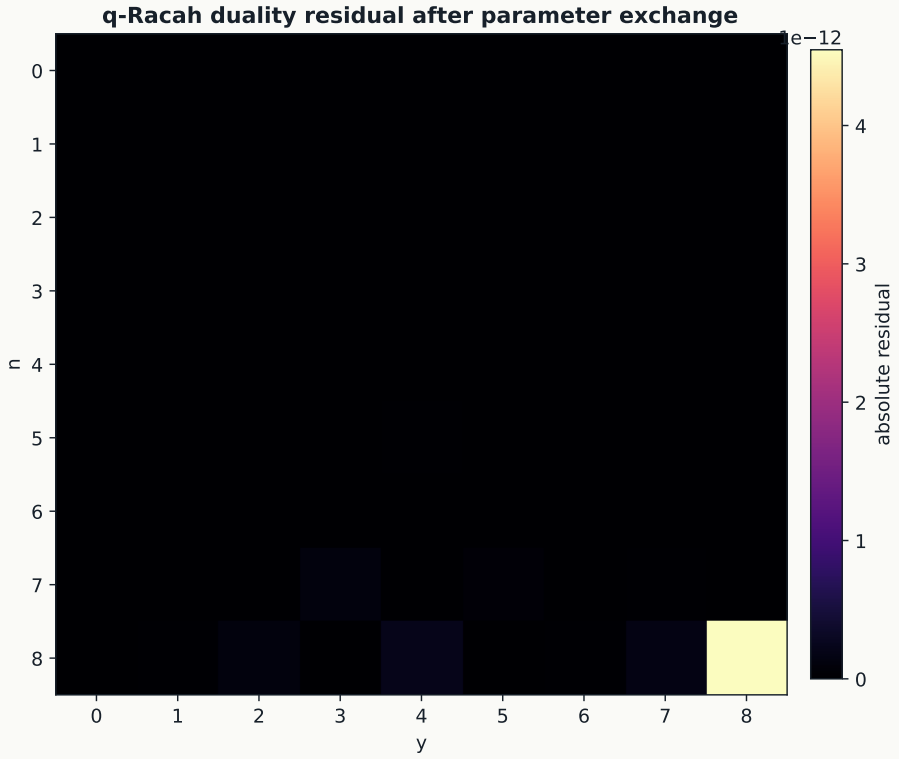


Figure 13.2: Duality residual. The parameter-exchanged degree-coordinate identity holds at floating-point precision.

Theorem 13.2 (Termwise classical limit)**.** *Set*

$$\alpha = q^A, \quad \beta = q^B, \quad \gamma = q^G, \quad \delta = q^D.$$

For fixed n and y,

$$\boxed{\lim_{q\to 1^-} R_n(\mu(y); q^A, q^B, q^G, q^D \mid q) = {}_4F_3\left(\begin{matrix}-n, n+A+B+1, -y, y+G+D+1 \\ A+1, B+D+1, G+1\end{matrix}; 1\right).}$$

Proof. The q-series is finite. For every fixed integer r,

$$\frac{(q^x;q)_r}{(1-q)^r} \longrightarrow (x)_r.$$

Apply this limit to each q-Pochhammer factor in every summand and cancel the equal powers of $1-q$. □

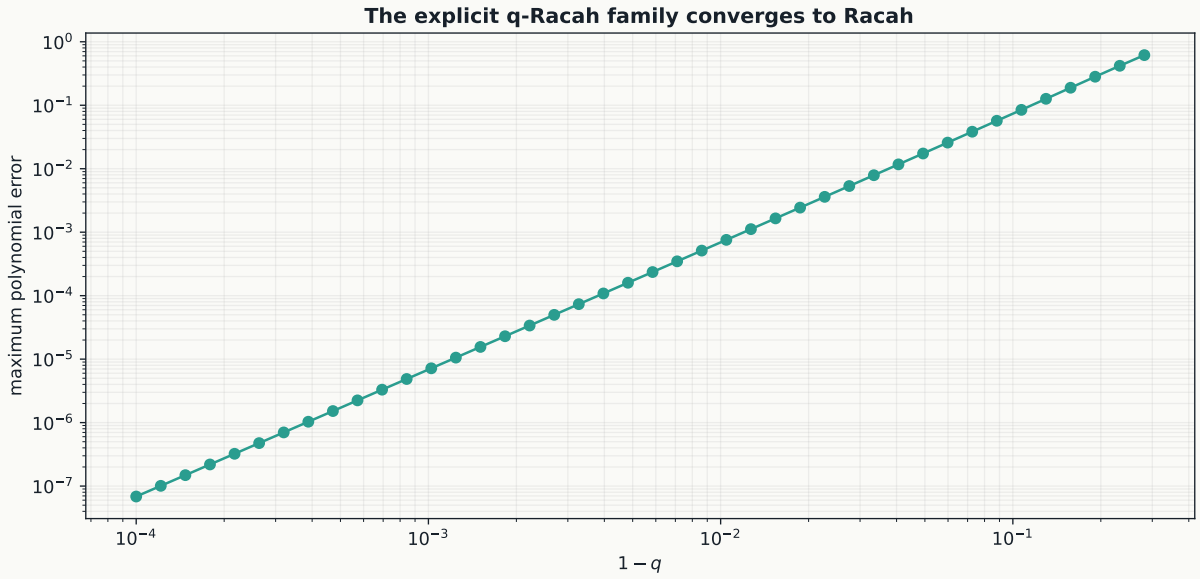


Figure 13.3: q-Racah to Racah. The finite basic-hypergeometric polynomial converges termwise to its ordinary hypergeometric counterpart.

Chapter 14

Askey-Wilson transfer function

For $z \in \mathbb{C}^{\times}$, define the normalized Askey-Wilson function

$$\boxed{\mathcal{A}_n(z) = {}_4\phi_3\left(\begin{matrix} q^{-n}, abcdq^{n-1}, az, az^{-1} \\ ab, ac, ad \end{matrix}; q, q\right).}$$

Define the q-difference operator

$$(\mathcal{L}f)(z) = A(z)(f(qz) - f(z)) + A(z^{-1})(f(q^{-1}z) - f(z)) + \left(1 + \frac{abcd}{q}\right) f(z),$$

where

$$A(z) = \frac{(1-az)(1-bz)(1-cz)(1-dz)}{(1-z^2)(1-qz^2)}.$$

Theorem 14.1 (Askey-Wilson eigenfunction law)**.** *For every nonnegative integer n,*

$$\boxed{\mathcal{L}\mathcal{A}_n = \left(q^{-n} + abcdq^{n-1}\right)\mathcal{A}_n.}$$

This is the continuous spectral lift of the finite q-Racah transfer system. The q-Racah family lives on a finite quadratic q-lattice. The Askey-Wilson family lives on the continuous variable $x = (z + z^{-1})/2$ and is governed by a second-order q-difference operator.

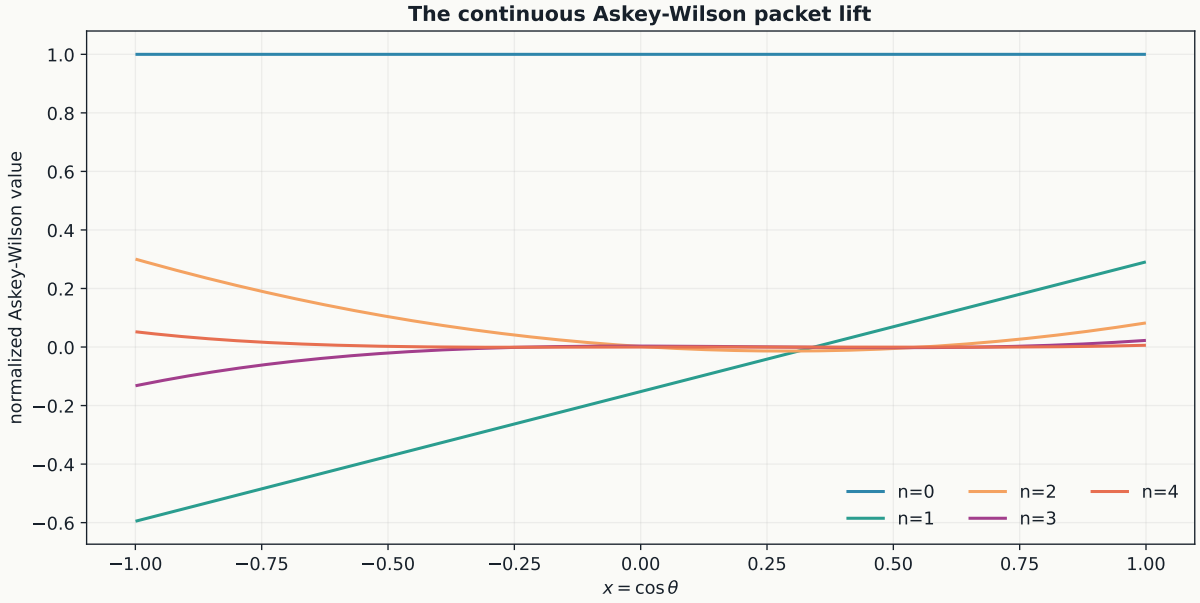


Figure 14.1: Askey-Wilson packet functions. Several degrees are plotted on the unit-circle coordinate $z = e^{i\theta}$.

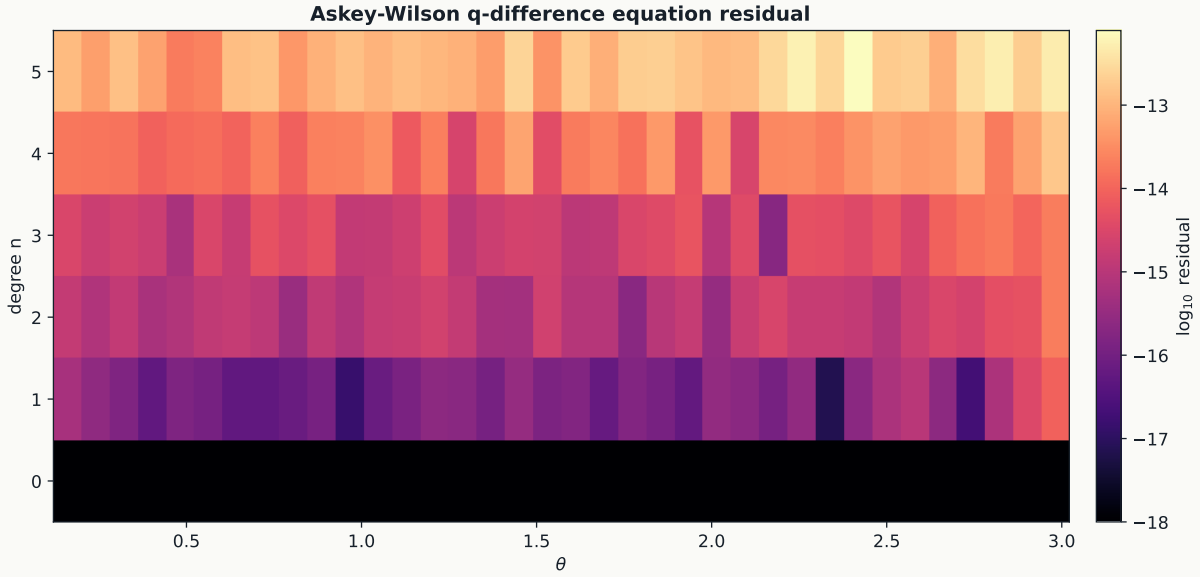


Figure 14.2: q-difference certificate. Direct substitution of the generated functions into the Askey-Wilson operator leaves only numerical roundoff.

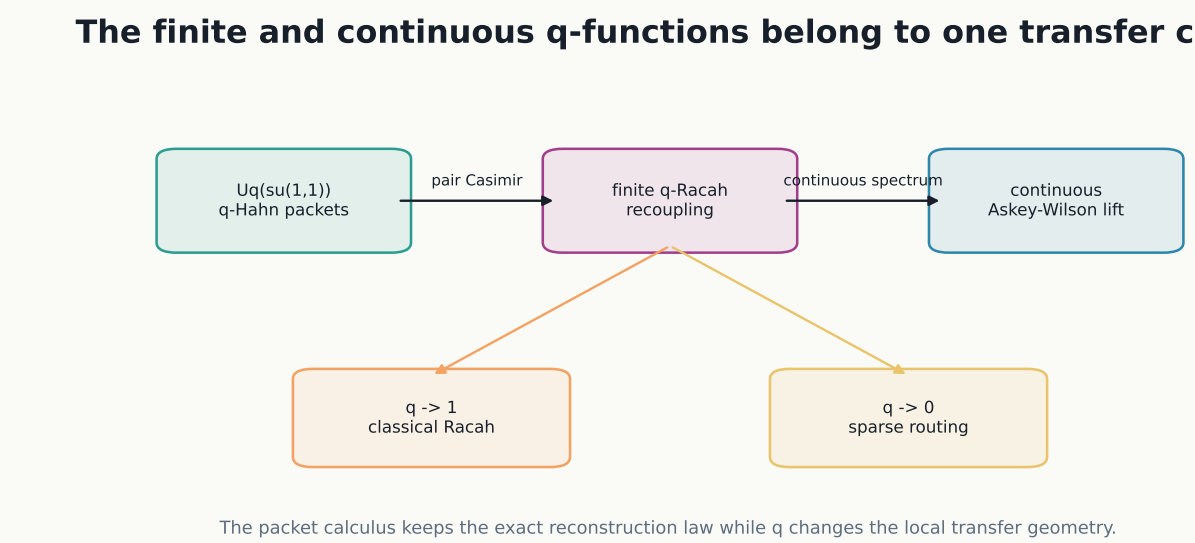


Figure 14.3: Finite and continuous q-functions. q-Hahn packets generate coupled bases, q-Racah functions generate finite recoupling, and Askey-Wilson functions provide the continuous spectral lift.

Part VIII

Quantum and computational interpretation

Chapter 15

A reversible q-shape module

The q-Racah transform can be interpreted as a finite coherent basis change. The signed amplitude matrix is reversible. Squaring the entries and measuring the target basis produces the bistochastic channel.

The deformation parameter can be treated as a compile-time or control-time variable. A program may request:

1. a lowest-weight q-Hahn seed,
2. a left or right pair-Casimir atlas,
3. a signed q-Racah transform,
4. its measured bistochastic kernel,
5. a q-deformation tangent generator,
6. a four-body associator certificate,
7. a finite Givens circuit.

This module can coexist with the logarithmic number Hamiltonian

$$H\,|n\rangle = \log n\,|n\rangle$$

used in the attached zeta-governed quantum dynamics. The logarithmic system supplies arithmetic time phases. The q-Racah system supplies finite shape coupling. Their tensor product is mathematically legitimate, but no theorem here identifies the q-Casimir spectrum with the nontrivial zeta zeros.

Chapter 16

What the deformation adds

The undeformed theory already has exact Racah recoupling and pentagon coherence. The q-coordinate adds four further layers.

First, it creates a continuous path between distinct packet-routing geometries.

Second, it turns basis sensitivity into the explicit skew generator $A_N(\theta)$.

Third, it produces q-dependent bistochastic measurement channels whose mixing behavior can be studied independently of coherent reversibility.

Fourth, it places the finite q-Racah packet and the continuous Askey-Wilson function inside one q-difference hierarchy.

The paper's strongest new operational object is therefore not a renamed polynomial. It is the complete tuple

$$\boxed{\left(\mathcal{M}_N, \mathcal{C}_{12}, \mathcal{C}_{23}, \mathcal{R}_N(q), P_N(q), A_N(\theta), \{U_r(q)\}_{r=0}^{4}\right).}$$

This tuple retains the object space, the two coordinate atlases, exact synthesis, the measured channel, the deformation tangent, and the associator loop.

Part IX

Verification and formulas

Chapter 17

Formula registry

Object	Exact formula or certificate
Symmetric q-number	$[x]_q = (q^x - q^{-x})/(q - q^{-1})$
Local q-algebra	$[K_-, K_+] = [2K_0]_q$
Coproduct	$\Delta K_\pm = K_\pm \otimes q^{K_0} + q^{-K_0} \otimes K_\pm$
q-Casimir	$\mathcal{C}_q = [K_0]_q\,[K_0 - 1]_q - K_+K_-$
Scalar Casimir	$\mathcal{C}_q = [\kappa]_q\,[\kappa - 1]_q\,\mathrm{I}$
Multiplicity dimension	$\dim \mathcal{M}_N^{(m)} = \binom{N+m-2}{m-2}$

Object	Exact formula or certificate
q-Hahn ratio	$\frac{c_{n+1}}{c_n} = -q^{-(N-1+\kappa_1+\kappa_2)} \times \sqrt{\frac{[N-n]_q\,[2\kappa_2+N-n-1]_q}{[n+1]_q\,[2\kappa_1+n]_q}}$
Pair spectrum	$\lambda_j^{ab} = [\kappa_a+\kappa_b+j]_q\,[\kappa_a+\kappa_b+j-1]_q$
Recoupling	$\mathcal{R}_N = L_N^{\mathsf{T}} R_N$
Bistochastic kernel	$P_{jm} = \lvert\mathcal{R}_{jm}\rvert^2$
Deformation generator	$A_N = \mathcal{R}_N^{\mathsf{T}}\partial_\theta \mathcal{R}_N$, $A_N^{\mathsf{T}} = -A_N$
Pentagon	$U_{01}U_{12}U_{23}U_{34}U_{40} = \mathrm{I}$
q-Racah duality	$R_n(y;\alpha,\beta,\gamma,\delta) = R_y(n;\gamma,\delta,\alpha,\beta)$
Askey-Wilson law	$\mathcal{L}\mathcal{A}_n = (q^{-n} + abcdq^{n-1})\mathcal{A}_n$
Givens bound	$L \le d(d-1)/2$

Chapter 18

Numerical certificate table

Check	Result	Largest recorded residual or witness
Local q-algebra	pass	2.05×10^{-12}
q-Hahn lowest vector	pass	2.22×10^{-16}
Scalar q-Casimir	pass	5.88×10^{-15}
q-Racah orthogonality	pass	2.22×10^{-15}
Pair spectrum	pass	4.55×10^{-13}
Opposing-Casimir tridiagonality	pass	5.21×10^{-12}
Closed q-beam splitter	pass	2.22×10^{-16}
Bistochastic kernel	pass	2.00×10^{-15}
Entropy nondecrease for fixed kernel	pass	no violation in 80 trials
One-excitation entropy monotonicity	pass	no negative increment
Higher-level monotonicity control	pass	decrement -7.78×10^{-3} found
Matrix classical limit	pass	2.82×10^{-7}
q-Hahn classical limit	pass	4.28×10^{-6}
Routing limit	pass	1.00×10^{-5}
Skew generator	pass	1.48×10^{-8} raw gauge residual
Tangent equation	pass	5.92×10^{-9}
Four-tree commutation	pass	7.96×10^{-13}
Joint diagonalization	pass	7.33×10^{-14}
q-pentagon	pass	3.22×10^{-15}
Givens compilation	pass	8.88×10^{-16}
q-Racah duality	pass	2.84×10^{-14}
q-Racah polynomial limit	pass	4.27×10^{-6}
Askey-Wilson q-difference equation	pass	5.91×10^{-13}

The independent test suite repeats twenty properties through direct calls to the mathematical functions. It does not merely read the built-in check ledger.

Chapter 19

Research directions and boundaries

The present paper suggests several focused questions.

1. Determine closed shape-parameter formulas for the full q-Racah overlap in the symmetric q-number convention used here.
2. Characterize the deformation metric $g_N(\theta)$ asymptotically as N grows.
3. Identify the precise packet conditions under which row entropy is monotone in q.
4. Extend the finite q-Casimir construction to continuous-series Askey-Wilson transforms.
5. Develop a q-adic or root-of-unity packet theory where semisimplicity may fail.
6. Compile the associator loop with locality constraints rather than unrestricted Givens rotations.
7. Couple the q-shape module to the logarithmic number Hamiltonian and determine which observables remain exactly factorized.

The first five items are special-function and representation-theoretic questions. The last two are architecture and quantum-model questions. None is solved by the numerical figures alone.

Chapter 20

Conclusion

The q-deformation does not discard the earlier packet calculus. It enriches it.

The seed is still a positive shape packet. The object space is still resolved by a lowest-weight projector. Intermediate Casimirs still create exact coordinate atlases. The recoupling matrix still supplies analysis and synthesis. The squared entries still define a measurement channel. Four coupling trees still close around a pentagon. The finite transform still compiles into two-coordinate rotations.

What changes is the geometry of every layer. q alters the ladder, pair spectra, packet weights, recoupling amplitudes, stochastic mixing, and associator edges. The q-Racah and Askey-Wilson families are the scalar special functions that record this deformation.

The result is a complete q-packet module:

$$\boxed{\text{q-Hahn seed} \longrightarrow \text{q-Casimir atlases} \longrightarrow \text{q-Racah transform} \longrightarrow \text{q-pentagon} \longrightarrow \text{Askey-Wilson lift.}}$$

The classical theory is recovered at $q = 1$. Selected packet transforms acquire sparse routing limits as q approaches zero. Between those endpoints, the skew generator provides a local differential description of the changing coordinate system.

Chapter A

Selected numerical tables

Table A.1: Equal-shape one-excitation q-beam splitter.

q	c_q	s_q	row entropy	spectral gap
0.10	0.099010	0.995086	0.055094	0.019606
0.25	0.235294	0.971924	0.214014	0.110727
0.50	0.400000	0.916515	0.439670	0.320000
0.75	0.480000	0.877268	0.539759	0.460800
1.00	0.500000	0.866025	0.562335	0.500000

Table A.2: Four-body q-pentagon residuals at $N = 4$.

q	multiplicity dimension	maximum residual
0.50	15	3.22×10^{-15}
0.70	15	4.00×10^{-15}
0.85	15	3.55×10^{-15}
1.00	15	3.44×10^{-15}

Chapter B

Complete Python implementation

The following source generates every figure, numerical table, built-in check, and manifest used in this paper.

```
from __future__ import annotations

import csv
import hashlib
import json
import math
from dataclasses import dataclass
from functools import lru_cache
from itertools import product
from pathlib import Path
from typing import Iterable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, FancyArrowPatch, FancyBboxPatch, Polygon
import numpy as np
from scipy.linalg import null_space

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT / "figures"
RESULTS = ROOT / "results"
TABLES = ROOT / "tables"
for directory in (FIGURES, RESULTS, TABLES):
    directory.mkdir(parents=True, exist_ok=True)

```

```
INK = "#17202A"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
MAGENTA = "#A23E8C"
RED = "#E76F51"
YELLOW = "#E9C46A"
PURPLE = "#6C5CE7"
MUTED = "#5D6D7E"
PAPER = "#FAFAF7"
GRID = "#D5D8DC"
WHITE = "#FFFFFF"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, RED, YELLOW, PURPLE]

plt.rcParams.update(
    {
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "font.size": 10.5,
        "axes.titlesize": 12.2,
        "axes.titleweight": "bold",
        "legend.frameon": False,
        "savefig.bbox": "tight",
        "savefig.facecolor": PAPER,
    }
)


def jsonable(value):
    """Convert NumPy scalar containers into JSON-safe Python objects."""
    if isinstance(value, np.generic):
        return value.item()
    if isinstance(value, dict):
        return {str(key): jsonable(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [jsonable(item) for item in value]
    return value


def save_figure(fig: plt.Figure, stem: str) -> None:
    fig.savefig(FIGURES / f"{stem}.pdf")
    fig.savefig(FIGURES / f"{stem}.png", dpi=220)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.6, 6.1)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    return fig, ax


def box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 10.5,
) -> None:
    patch = FancyBboxPatch(
        xy,
        width,
```

```
        height,
        boxstyle="round,pad=.012,rounding_size=.018",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.6,
    )
    ax.add_patch(patch)
    ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=fontsize)


def arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
    text_offset: tuple[float, float] = (0.0, 0.035),
) -> None:
    patch = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13, linewidth=1.5, color=color)
    ax.add_patch(patch)
    if text:
        ax.text(
            (start[0] + end[0]) / 2 + text_offset[0],
            (start[1] + end[1]) / 2 + text_offset[1],
            text,
            ha="center",
            va="center",
            fontsize=9.5,
            color=color,
        )


# ---------------------------------------------------------------------------
# q-arithmetic and positive discrete-series packet spaces
# ---------------------------------------------------------------------------


def q_number(x: float, q: float) -> float:
    if q <= 0:
        raise ValueError("q must be positive")
    if abs(q - 1.0) < 1e-12:
        return float(x)
    theta = math.log(q)
    return float(math.sinh(x * theta) / math.sinh(theta))


def q_factorial(n: int, q: float) -> float:
    result = 1.0
    for j in range(1, n + 1):
        result *= q_number(float(j), q)
    return result


def q_binomial(n: int, r: int, q: float) -> float:
    if r < 0 or r > n:
        return 0.0
    return q_factorial(n, q) / (q_factorial(r, q) * q_factorial(n - r, q))


def q_rising(x: float, n: int, q: float) -> float:
    result = 1.0
    for j in range(n):
        result *= q_number(x + j, q)
    return result


@lru_cache(maxsize=None)
def compositions(total: int, parts: int) -> tuple[tuple[int, ...], ...]:
    if total < 0:
        return tuple()
    if parts == 1:
```

```
        return ((total,),)
    out: list[tuple[int, ...]] = []
    for first in range(total + 1):
        for rest in compositions(total - first, parts - 1):
            out.append((first,) + rest)
    return tuple(out)


def excitation_dimension(total: int, factors: int) -> int:
    return math.comb(total + factors - 1, factors - 1)


def multiplicity_dimension(total: int, factors: int) -> int:
    if factors < 2:
        return 1
    return math.comb(total + factors - 2, factors - 2)


def q_local_matrices(dimension: int, kappa: float, q: float) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    k0 = np.diag([n + kappa for n in range(dimension)])
    kp = np.zeros((dimension, dimension), dtype=float)
    km = np.zeros((dimension, dimension), dtype=float)
    for n in range(dimension - 1):
        coefficient = math.sqrt(q_number(n + 1.0, q) * q_number(2.0 * kappa + n, q))
        kp[n + 1, n] = coefficient
        km[n, n + 1] = coefficient
    return k0, kp, km


def q_lowering_matrix(
    total: int,
    kappas: Sequence[float],
    q: float,
    subset: Iterable[int] | None = None,
) -> np.ndarray:
    factors = len(kappas)
    active = list(range(factors)) if subset is None else [int(value) for value in subset]
    basis_now = compositions(total, factors)
    if total == 0:
        return np.zeros((0, len(basis_now)), dtype=float)
    basis_previous = compositions(total - 1, factors)
    previous_index = {state: index for index, state in enumerate(basis_previous)}
    matrix = np.zeros((len(basis_previous), len(basis_now)), dtype=float)
    for column, state in enumerate(basis_now):
        for position, factor in enumerate(active):
            occupation = state[factor]
            if occupation == 0:
                continue
            coefficient = math.sqrt(
                q_number(float(occupation), q)
                * q_number(2.0 * kappas[factor] + occupation - 1.0, q)
            )
            exponent = 0.0
            for left in active[:position]:
                exponent -= state[left] + kappas[left]
            for right in active[position + 1 :]:
                exponent += state[right] + kappas[right]
            coefficient *= q**exponent
            target = list(state)
            target[factor] -= 1
            matrix[previous_index[tuple(target)], column] += coefficient
    return matrix


def q_casimir_matrix(total: int, kappas: Sequence[float], q: float, subset: Iterable[int]) -> np.ndarray:
    active = [int(value) for value in subset]
    basis = compositions(total, len(kappas))
    k0_values = np.array([sum(state[index] + kappas[index] for index in active) for state in basis])
    lowering = q_lowering_matrix(total, kappas, q, active)
    diagonal = np.array([q_number(value, q) * q_number(value - 1.0, q) for value in k0_values])
    return np.diag(diagonal) - lowering.T @ lowering
```

```

def q_multiplicity_frame(total: int, kappas: Sequence[float], q: float, atol: float = 1e-11) -> np.ndarray:
    if total == 0:
        return np.ones((1, 1), dtype=float)
    lowering = q_lowering_matrix(total, kappas, q)
    return null_space(lowering, rcond=atol)


def sign_fix_columns(matrix: np.ndarray) -> np.ndarray:
    result = np.array(matrix, dtype=float, copy=True)
    for column in range(result.shape[1]):
        row = int(np.argmax(np.abs(result[:, column])))
        if result[row, column] < 0:
            result[:, column] *= -1.0
    return result


def canonicalize_overlap(matrix: np.ndarray) -> np.ndarray:
    result = np.array(matrix, dtype=float, copy=True)
    for column in range(result.shape[1]):
        if result[0, column] < 0:
            result[:, column] *= -1.0
    for row in range(result.shape[0]):
        if result[row, 0] < 0:
            result[row, :] *= -1.0
    return result


def align_signed_matrix(reference: np.ndarray, candidate: np.ndarray, iterations: int = 8) -> np.ndarray:
    result = np.array(candidate, dtype=float, copy=True)
    for _ in range(iterations):
        for column in range(result.shape[1]):
            if float(np.dot(reference[:, column], result[:, column])) < 0:
                result[:, column] *= -1.0
        for row in range(result.shape[0]):
            if float(np.dot(reference[row, :], result[row, :])) < 0:
                result[row, :] *= -1.0
    return result


@dataclass
class QRecouplingData:
    total: int
    kappas: tuple[float, float, float]
    q: float
    multiplicity_frame: np.ndarray
    left_eigenvalues: np.ndarray
    right_eigenvalues: np.ndarray
    left_basis: np.ndarray
    right_basis: np.ndarray
    overlap: np.ndarray
    right_in_left: np.ndarray


def q_three_body_recoupling(total: int, kappas: Sequence[float], q: float) -> QRecouplingData:
    if len(kappas) != 3:
        raise ValueError("q_three_body_recoupling requires three shapes")
    kappas_tuple = tuple(float(value) for value in kappas)
    frame = q_multiplicity_frame(total, kappas_tuple, q)
    c12 = frame.T @ q_casimir_matrix(total, kappas_tuple, q, (0, 1)) @ frame
    c23 = frame.T @ q_casimir_matrix(total, kappas_tuple, q, (1, 2)) @ frame
    left_values, left_vectors = np.linalg.eigh(c12)
    right_values, right_vectors = np.linalg.eigh(c23)
    left_vectors = sign_fix_columns(left_vectors)
    right_vectors = sign_fix_columns(right_vectors)
    left_basis = frame @ left_vectors
    right_basis = frame @ right_vectors
    overlap = canonicalize_overlap(left_basis.T @ right_basis)
    right_in_left = overlap @ np.diag(right_values) @ overlap.T
    return QRecouplingData(
```

```
        total=total,
        kappas=kappas_tuple,
        q=float(q),
        multiplicity_frame=frame,
        left_eigenvalues=left_values,
        right_eigenvalues=right_values,
        left_basis=left_basis,
        right_basis=right_basis,
        overlap=overlap,
        right_in_left=right_in_left,
    )


def q_pair_spectrum(total: int, kappa_a: float, kappa_b: float, q: float) -> np.ndarray:
    return np.array(
        [
            q_number(kappa_a + kappa_b + j, q)
            * q_number(kappa_a + kappa_b + j - 1.0, q)
            for j in range(total + 1)
        ]
    )


def q_two_body_lowest_coefficients(total: int, kappa_1: float, kappa_2: float, q: float) -> np.ndarray:
    tilt = q ** (-(total - 1.0 + kappa_1 + kappa_2))
    coefficients = []
    for n in range(total + 1):
        weight = q_binomial(total, n, q)
        weight *= q_rising(2.0 * kappa_2 + total - n, n, q)
        weight /= q_rising(2.0 * kappa_1, n, q)
        coefficients.append(((-tilt) ** n) * math.sqrt(weight))
    vector = np.array(coefficients, dtype=float)
    return vector / np.linalg.norm(vector)


def equal_shape_one_excitation_matrix(q: float) -> np.ndarray:
    denominator = 1.0 + q * q
    cosine = q / denominator
    sine = math.sqrt(1.0 + q * q + q**4) / denominator
    return np.array([[cosine, sine], [sine, -cosine]], dtype=float)


def tridiagonal_residual(matrix: np.ndarray) -> float:
    residual = np.array(matrix, copy=True)
    for row in range(residual.shape[0]):
        for column in range(residual.shape[1]):
            if abs(row - column) <= 1:
                residual[row, column] = 0.0
    return float(np.max(np.abs(residual)))


def row_entropies(probability_matrix: np.ndarray) -> np.ndarray:
    safe = np.where(probability_matrix > 0, probability_matrix, 1.0)
    return -np.sum(np.where(probability_matrix > 0, probability_matrix * np.log(safe), 0.0), axis=1)


def doubly_stochastic_residual(matrix: np.ndarray) -> float:
    return float(
        max(
            np.max(np.abs(matrix.sum(axis=0) - 1.0)),
            np.max(np.abs(matrix.sum(axis=1) - 1.0)),
            np.max(np.maximum(-matrix, 0.0)),
        )
    )


def spectral_mixing_gap(probability_matrix: np.ndarray) -> float:
    eigenvalues = np.linalg.eigvals(probability_matrix)
    magnitudes = sorted((abs(value) for value in eigenvalues), reverse=True)
    if len(magnitudes) < 2:
        return 1.0
```

```
    return float(1.0 - magnitudes[1])


# ---------------------------------------------------------------------------
# q-deformation flow
# ---------------------------------------------------------------------------


def deformation_generator(total: int, kappas: Sequence[float], theta: float, step: float = 2e-5) -> dict[str, np.
     ndarray | float]:
    q0 = math.exp(-theta)
    reference = q_three_body_recoupling(total, kappas, q0).overlap
    plus = q_three_body_recoupling(total, kappas, math.exp(-(theta + step))).overlap
    minus = q_three_body_recoupling(total, kappas, math.exp(-(theta - step))).overlap
    plus = align_signed_matrix(reference, plus)
    minus = align_signed_matrix(reference, minus)
    derivative = (plus - minus) / (2.0 * step)
    raw = reference.T @ derivative
    generator = 0.5 * (raw - raw.T)
    skew_residual = float(np.max(np.abs(raw + raw.T)))
    tangent_residual = float(np.max(np.abs(derivative - reference @ generator)))
    metric = 0.5 * float(np.sum(generator * generator))
    return {
        "q": q0,
        "matrix": reference,
        "derivative": derivative,
        "generator": generator,
        "skew_residual": skew_residual,
        "tangent_residual": tangent_residual,
        "metric": metric,
    }


# ---------------------------------------------------------------------------
# Four-body q-coupling trees and pentagon coherence
# ---------------------------------------------------------------------------


FOUR_TREES: tuple[tuple[str, tuple[tuple[int, ...], tuple[int, ...]]], ...] = (
    ("(((12)3)4)", ((0, 1), (0, 1, 2))),
    ("((1(23))4)", ((1, 2), (0, 1, 2))),
    ("(1((23)4))", ((1, 2), (1, 2, 3))),
    ("(1(2(34)))", ((2, 3), (1, 2, 3))),
    ("((12)(34))", ((0, 1), (2, 3))),
)


@dataclass
class QTreeBasisData:
    name: str
    subsets: tuple[tuple[int, ...], ...]
    basis: np.ndarray
    joint_diagonals: tuple[np.ndarray, ...]
    commutator_residual: float
    diagonalization_residual: float


def q_joint_tree_basis(
    total: int,
    kappas: Sequence[float],
    q: float,
    name: str,
    subsets: Sequence[Sequence[int]],
) -> QTreeBasisData:
    frame = q_multiplicity_frame(total, kappas, q)
    restricted = [frame.T @ q_casimir_matrix(total, kappas, q, subset) @ frame for subset in subsets]
    commutator = 0.0
    for i in range(len(restricted)):
        for j in range(i + 1, len(restricted)):
            commutator = max(
                commutator,
```

```
                float(np.max(np.abs(restricted[i] @ restricted[j] - restricted[j] @ restricted[i]))),
            )
    irrational_weights = [math.sqrt(2.0), math.sqrt(3.0), math.sqrt(5.0)]
    combined = sum(weight * matrix for weight, matrix in zip(irrational_weights, restricted))
    _, vectors = np.linalg.eigh(combined)
    vectors = sign_fix_columns(vectors)
    diagonals = tuple(vectors.T @ matrix @ vectors for matrix in restricted)
    tuples = []
    for column in range(vectors.shape[1]):
        key = tuple(round(float(diagonal[column, column]), 10) for diagonal in diagonals)
        tuples.append((key, column))
    order = [column for _, column in sorted(tuples, key=lambda item: item[0])]
    vectors = vectors[:, order]
    diagonals = tuple(vectors.T @ matrix @ vectors for matrix in restricted)
    diagonalization_residual = max(
        float(np.max(np.abs(diagonal - np.diag(np.diag(diagonal))))) for diagonal in diagonals
    )
    return QTreeBasisData(
        name=name,
        subsets=tuple(tuple(int(index) for index in subset) for subset in subsets),
        basis=frame @ vectors,
        joint_diagonals=diagonals,
        commutator_residual=commutator,
        diagonalization_residual=diagonalization_residual,
    )


def q_four_body_tree_bases(total: int, kappas: Sequence[float], q: float) -> list[QTreeBasisData]:
    if len(kappas) != 4:
        raise ValueError("q_four_body_tree_bases requires four shapes")
    return [q_joint_tree_basis(total, kappas, q, name, subsets) for name, subsets in FOUR_TREES]


def q_pentagon_product(tree_bases: Sequence[QTreeBasisData]) -> tuple[np.ndarray, list[np.ndarray]]:
    dimension = tree_bases[0].basis.shape[1]
    product_matrix = np.eye(dimension)
    overlaps: list[np.ndarray] = []
    for index in range(len(tree_bases)):
        left = tree_bases[index].basis
        right = tree_bases[(index + 1) % len(tree_bases)].basis
        overlap = left.T @ right
        overlaps.append(overlap)
        product_matrix = product_matrix @ overlap
    return product_matrix, overlaps


# ---------------------------------------------------------------------------
# q-hypergeometric q-Racah and Askey-Wilson functions
# ---------------------------------------------------------------------------


def q_pochhammer(a: complex, q: float, n: int) -> complex:
    result = 1.0 + 0.0j
    for j in range(n):
        result *= 1.0 - a * (q**j)
    return result


def q_racah_polynomial(
    n: int,
    y: int,
    alpha: float,
    beta: float,
    gamma: float,
    delta: float,
    q: float,
) -> complex:
    result = 0.0 + 0.0j
    for ell in range(n + 1):
        numerator = q_pochhammer(q ** (-n), q, ell)
        numerator *= q_pochhammer(alpha * beta * q ** (n + 1), q, ell)
```

```
        numerator *= q_pochhammer(q ** (-y), q, ell)
        numerator *= q_pochhammer(gamma * delta * q ** (y + 1), q, ell)
        denominator = q_pochhammer(alpha * q, q, ell)
        denominator *= q_pochhammer(beta * delta * q, q, ell)
        denominator *= q_pochhammer(gamma * q, q, ell)
        denominator *= q_pochhammer(q, q, ell)
        result += numerator / denominator * q**ell
    return result


def rising(a: float, n: int) -> float:
    result = 1.0
    for j in range(n):
        result *= a + j
    return result


def classical_racah_polynomial(n: int, y: int, alpha: float, beta: float, gamma: float, delta: float) -> float:
    result = 0.0
    for ell in range(n + 1):
        numerator = rising(-float(n), ell)
        numerator *= rising(n + alpha + beta + 1.0, ell)
        numerator *= rising(-float(y), ell)
        numerator *= rising(y + gamma + delta + 1.0, ell)
        denominator = rising(alpha + 1.0, ell)
        denominator *= rising(beta + delta + 1.0, ell)
        denominator *= rising(gamma + 1.0, ell)
        denominator *= math.factorial(ell)
        result += numerator / denominator
    return result


def askey_wilson_r(
    n: int,
    z: complex,
    a: float,
    b: float,
    c: float,
    d: float,
    q: float,
) -> complex:
    result = 0.0 + 0.0j
    for ell in range(n + 1):
        numerator = q_pochhammer(q ** (-n), q, ell)
        numerator *= q_pochhammer(a * b * c * d * q ** (n - 1), q, ell)
        numerator *= q_pochhammer(a * z, q, ell)
        numerator *= q_pochhammer(a / z, q, ell)
        denominator = q_pochhammer(a * b, q, ell)
        denominator *= q_pochhammer(a * c, q, ell)
        denominator *= q_pochhammer(a * d, q, ell)
        denominator *= q_pochhammer(q, q, ell)
        result += numerator / denominator * q**ell
    return result


def askey_wilson_operator(
    function,
    z: complex,
    a: float,
    b: float,
    c: float,
    d: float,
    q: float,
) -> complex:
    first = ((1 - a * z) * (1 - b * z) * (1 - c * z) * (1 - d * z))
    first /= (1 - z * z) * (1 - q * z * z)
    inverse = 1.0 / z
    second = ((1 - a * inverse) * (1 - b * inverse) * (1 - c * inverse) * (1 - d * inverse))
    second /= (1 - inverse * inverse) * (1 - q * inverse * inverse)
    return (
        first * (function(q * z) - function(z))
```

```
        + second * (function(z / q) - function(z))
        + (1.0 + a * b * c * d / q) * function(z)
    )


# ---------------------------------------------------------------------------
# Givens compilation
# ---------------------------------------------------------------------------


@dataclass
class GivensRotation:
    upper: int
    lower: int
    cosine: float
    sine: float


def givens_decompose(orthogonal: np.ndarray, atol: float = 1e-14) -> tuple[list[GivensRotation], np.ndarray]:
    matrix = np.array(orthogonal, dtype=float, copy=True)
    dimension = matrix.shape[0]
    rotations: list[GivensRotation] = []
    for column in range(dimension):
        for lower in range(dimension - 1, column, -1):
            upper = lower - 1
            a = matrix[upper, column]
            b = matrix[lower, column]
            radius = math.hypot(a, b)
            if radius < atol:
                continue
            cosine = a / radius
            sine = b / radius
            row_upper = cosine * matrix[upper, :] + sine * matrix[lower, :]
            row_lower = -sine * matrix[upper, :] + cosine * matrix[lower, :]
            matrix[upper, :] = row_upper
            matrix[lower, :] = row_lower
            rotations.append(GivensRotation(upper, lower, cosine, sine))
    return rotations, np.diag(np.diag(matrix))


def givens_reconstruct(rotations: Sequence[GivensRotation], diagonal: np.ndarray) -> np.ndarray:
    matrix = np.array(diagonal, dtype=float, copy=True)
    for rotation in reversed(rotations):
        upper = rotation.upper
        lower = rotation.lower
        cosine = rotation.cosine
        sine = rotation.sine
        row_upper = cosine * matrix[upper, :] - sine * matrix[lower, :]
        row_lower = sine * matrix[upper, :] + cosine * matrix[lower, :]
        matrix[upper, :] = row_upper
        matrix[lower, :] = row_lower
    return matrix


# ---------------------------------------------------------------------------
# Checks and result tables
# ---------------------------------------------------------------------------


def run_checks() -> dict[str, dict[str, float | int | str | bool]]:
    checks: dict[str, dict[str, float | int | str | bool]] = {}

    q = 0.73
    kappa = 0.8
    dimension = 12
    k0, kp, km = q_local_matrices(dimension, kappa, q)
    commutator = km @ kp - kp @ km
    target = np.diag([q_number(2.0 * (n + kappa), q) for n in range(dimension)])
    algebra_residual = float(np.max(np.abs(commutator[:-1, :-1] - target[:-1, :-1])))
    checks["q_algebra"] = {"passed": algebra_residual < 2e-10, "residual": algebra_residual}

```

```
    total = 5
    kappas_two = (0.7, 1.1)
    coefficients = q_two_body_lowest_coefficients(total, *kappas_two, q)
    lowest_residual = float(np.max(np.abs(q_lowering_matrix(total, kappas_two, q) @ coefficients)))
    checks["q_hahn_lowest_weight"] = {"passed": lowest_residual < 2e-10, "residual": lowest_residual}

    casimir = q_casimir_matrix(6, (kappa,), q, (0,))
    scalar = q_number(kappa, q) * q_number(kappa - 1.0, q)
    casimir_residual = float(np.max(np.abs(casimir - scalar * np.eye(casimir.shape[0]))))
    checks["q_casimir_scalar"] = {"passed": casimir_residual < 2e-10, "residual": casimir_residual}

    data = q_three_body_recoupling(8, (0.7, 1.1, 0.9), q)
    orthogonality = float(np.max(np.abs(data.overlap @ data.overlap.T - np.eye(9))))
    checks["q_racah_orthogonality"] = {"passed": orthogonality < 2e-10, "residual": orthogonality}

    left_theory = q_pair_spectrum(8, 0.7, 1.1, q)
    spectrum_residual = float(np.max(np.abs(data.left_eigenvalues - left_theory)))
    checks["q_pair_spectrum"] = {"passed": spectrum_residual < 2e-9, "residual": spectrum_residual}

    tri_residual = tridiagonal_residual(data.right_in_left)
    checks["q_racah_tridiagonal"] = {"passed": tri_residual < 3e-9, "residual": tri_residual}

    exact = equal_shape_one_excitation_matrix(q)
    computed = q_three_body_recoupling(1, (0.5, 0.5, 0.5), q).overlap
    beam_residual = float(np.max(np.abs(computed - exact)))
    checks["q_beam_splitter_closed_form"] = {"passed": beam_residual < 2e-10, "residual": beam_residual}

    probability = data.overlap**2
    stochastic_residual = doubly_stochastic_residual(probability)
    checks["q_bistochastic_kernel"] = {"passed": stochastic_residual < 2e-10, "residual": stochastic_residual}

    rng = np.random.default_rng(801)
    entropy_fail = 0.0
    for _ in range(80):
        vector = rng.random(probability.shape[0])
        vector /= vector.sum()
        before = -float(np.sum(vector * np.log(vector)))
        after_vector = vector @ probability
        after = -float(np.sum(after_vector * np.log(after_vector)))
        entropy_fail = max(entropy_fail, before - after)
    checks["q_entropy_non_decrease"] = {"passed": entropy_fail < 2e-12, "residual": entropy_fail}

    q_values = np.linspace(0.12, 1.0, 80)
    entropy_one = []
    for value in q_values:
        matrix = equal_shape_one_excitation_matrix(float(value))
        entropy_one.append(row_entropies(matrix**2)[0])
    monotone_residual = float(max(0.0, -np.min(np.diff(entropy_one))))
    checks["one_excitation_entropy_monotone"] = {"passed": monotone_residual < 2e-10, "residual": monotone_residual}

    entropy_middle = []
    for value in q_values:
        matrix = q_three_body_recoupling(3, (0.5, 0.5, 0.5), float(value)).overlap
        entropy_middle.append(row_entropies(matrix**2)[1])
    negative_control = float(np.min(np.diff(entropy_middle)))
    checks["higher_entropy_nonmonotone_control"] = {
        "passed": negative_control < -1e-5,
        "minimum_increment": negative_control,
    }

    classical = q_three_body_recoupling(8, (0.7, 1.1, 0.9), 1.0).overlap
    near = align_signed_matrix(classical, q_three_body_recoupling(8, (0.7, 1.1, 0.9), 0.9999).overlap)
    classical_limit = float(np.max(np.abs(near - classical)))
    checks["q_to_one_recoupling"] = {"passed": classical_limit < 2e-5, "residual": classical_limit}

    qh_classical = q_two_body_lowest_coefficients(7, 0.7, 1.1, 1.0)
    qh_near = q_two_body_lowest_coefficients(7, 0.7, 1.1, 0.999999)
    if float(np.dot(qh_classical, qh_near)) < 0:
        qh_near *= -1.0
    qh_limit = float(np.max(np.abs(qh_near - qh_classical)))
    checks["q_hahn_classical_limit"] = {"passed": qh_limit < 2e-5, "residual": qh_limit}
```

```
736
737     small_q = equal_shape_one_excitation_matrix(1e-5)
738     routing_target = np.array([[0.0, 1.0], [1.0, 0.0]])
739     routing_residual = float(np.max(np.abs(small_q - routing_target)))
740     checks["q_to_zero_routing"] = {"passed": routing_residual < 2e-5, "residual": routing_residual}
741
742     deformation = deformation_generator(6, (0.7, 1.1, 0.9), theta=0.35)
743     checks["deformation_generator_skew"] = {
744         "passed": float(deformation["skew_residual"]) < 2e-7,
745         "residual": float(deformation["skew_residual"]),
746     }
747     checks["deformation_tangent_equation"] = {
748         "passed": float(deformation["tangent_residual"]) < 2e-7,
749         "residual": float(deformation["tangent_residual"]),
750     }
751
752     four_kappas = (0.55, 0.8, 1.05, 1.3)
753     trees = q_four_body_tree_bases(4, four_kappas, 0.78)
754     commutator_residual = max(tree.commutator_residual for tree in trees)
755     diagonal_residual = max(tree.diagonalization_residual for tree in trees)
756     product_matrix, _ = q_pentagon_product(trees)
757     pentagon_residual = float(np.max(np.abs(product_matrix - np.eye(product_matrix.shape[0]))))
758     checks["q_tree_commutation"] = {"passed": commutator_residual < 2e-8, "residual": commutator_residual}
759     checks["q_tree_diagonalization"] = {"passed": diagonal_residual < 2e-8, "residual": diagonal_residual}
760     checks["q_pentagon"] = {"passed": pentagon_residual < 2e-8, "residual": pentagon_residual}
761
762     rotations, diagonal = givens_decompose(data.overlap)
763     reconstructed = givens_reconstruct(rotations, diagonal)
764     givens_residual = float(np.max(np.abs(reconstructed - data.overlap)))
765     checks["q_givens_compilation"] = {
766         "passed": givens_residual < 2e-10,
767         "residual": givens_residual,
768         "rotation_count": len(rotations),
769     }
770
771     q_poly = 0.73
772     degree_limit = 6
773     alpha_exp = 0.6
774     beta_exp = 0.8
775     gamma_exp = -degree_limit - 1.0
776     delta_exp = 0.4
777     alpha = q_poly**alpha_exp
778     beta = q_poly**beta_exp
779     gamma = q_poly**gamma_exp
780     delta = q_poly**delta_exp
781     duality_residual = 0.0
782     for n in range(degree_limit + 1):
783         for y in range(degree_limit + 1):
784             left = q_racah_polynomial(n, y, alpha, beta, gamma, delta, q_poly)
785             right = q_racah_polynomial(y, n, gamma, delta, alpha, beta, q_poly)
786             duality_residual = max(duality_residual, abs(left - right))
787     checks["q_racah_duality"] = {"passed": duality_residual < 2e-10, "residual": float(duality_residual)}
788
789     q_limit = 0.999
790     alpha = q_limit**alpha_exp
791     beta = q_limit**beta_exp
792     gamma = q_limit**gamma_exp
793     delta = q_limit**delta_exp
794     polynomial_limit = 0.0
795     for n in range(4):
796         for y in range(degree_limit + 1):
797             q_value = q_racah_polynomial(n, y, alpha, beta, gamma, delta, q_limit).real
798             classical_value = classical_racah_polynomial(n, y, alpha_exp, beta_exp, gamma_exp, delta_exp)
799             polynomial_limit = max(polynomial_limit, abs(q_value - classical_value))
800     checks["q_racah_polynomial_limit"] = {"passed": polynomial_limit < 2e-4, "residual": polynomial_limit}
801
802     aw_q = 0.7
803     aw_parameters = (0.2, 0.3, -0.25, 0.4)
804     aw_residual = 0.0
805     for n in range(6):
806         for theta in np.linspace(0.31, 2.72, 8):
```

```
            z = np.exp(1j * theta)
            function = lambda value, degree=n: askey_wilson_r(degree, value, *aw_parameters, aw_q)
            left = askey_wilson_operator(function, z, *aw_parameters, aw_q)
            eigenvalue = aw_q ** (-n) + np.prod(aw_parameters) * aw_q ** (n - 1)
            aw_residual = max(aw_residual, abs(left - eigenvalue * function(z)))
    checks["askey_wilson_difference_equation"] = {"passed": aw_residual < 2e-9, "residual": float(aw_residual)}

    passed = sum(1 for item in checks.values() if bool(item["passed"]))
    summary = {"checks": checks, "passed": passed, "total": len(checks), "all_passed": passed == len(checks)}
    (RESULTS / "checks.json").write_text(json.dumps(jsonable(summary), indent=2))
    return checks


# ---------------------------------------------------------------------------
# Figures
# ---------------------------------------------------------------------------


def generate_figures() -> list[str]:
    generated: list[str] = []

    fig, ax = diagram_axes((13.0, 6.3))
    ax.text(0.5, 0.94, "From classical shape packets to q-deformed recoupling", ha="center", fontsize=21, weight="
     bold")
    stages = [
        (0.03, 0.58, 0.16, 0.17, "q-number\nladder", BLUE),
        (0.23, 0.58, 0.16, 0.17, "q-Hahn\nlowest packet", TEAL),
        (0.43, 0.58, 0.16, 0.17, "q-Casimir\natlases", ORANGE),
        (0.63, 0.58, 0.16, 0.17, "q-Racah\ntransform", MAGENTA),
        (0.83, 0.58, 0.14, 0.17, "q-pentagon\ncoherence", RED),
    ]
    for x, y, w, h, label, color in stages:
        box(ax, (x, y), w, h, label, face=color + "1C", edge=color, fontsize=11.5)
    for left, right in zip(stages[:-1], stages[1:]):
        arrow(ax, (left[0] + left[2], left[1] + left[3] / 2), (right[0], right[1] + right[3] / 2))
    box(ax, (0.12, 0.18), 0.20, 0.16, "bistochastic\nmeasurement", face=YELLOW + "25", edge=YELLOW, fontsize=11.5)
    box(ax, (0.40, 0.18), 0.20, 0.16, "deformation\ngenerator", face=PURPLE + "1C", edge=PURPLE, fontsize=11.5)
    box(ax, (0.68, 0.18), 0.20, 0.16, "Askey-Wilson\ncontinuous lift", face=BLUE + "1C", edge=BLUE, fontsize=11.5)
    arrow(ax, (0.70, 0.58), (0.22, 0.34), color=MAGENTA)
    arrow(ax, (0.72, 0.58), (0.50, 0.34), color=PURPLE)
    arrow(ax, (0.88, 0.58), (0.78, 0.34), color=RED)
    ax.text(0.5, 0.07, "The deformation parameter changes the packet algebra while preserving exact analysis,
     synthesis, and associator closure.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "01_theory_pipeline")
    generated.append("01_theory_pipeline")

    x = np.linspace(0.0, 8.0, 300)
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    for q, color in zip((0.45, 0.65, 0.82, 0.95, 1.0), COLORS[:5]):
        ax.plot(x, [q_number(value, q) for value in x], color=color, linewidth=2.0, label=f"q={q:g}")
    ax.plot(x, x, color=INK, linestyle="--", linewidth=1.2, label="classical x")
    ax.set_xlabel("x")
    ax.set_ylabel(r"$[x]_q$")
    ax.set_title("Symmetric q-numbers deform the linear ladder")
    ax.grid(alpha=0.18)
    ax.legend(ncol=3)
    save_figure(fig, "02_q_number_deformation")
    generated.append("02_q_number_deformation")

    n = np.arange(0, 14)
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    for q, color in zip((0.45, 0.7, 0.9, 1.0), (BLUE, TEAL, ORANGE, MAGENTA)):
        values = [math.sqrt(q_number(value + 1.0, q) * q_number(value + 1.6, q)) for value in n]
        ax.plot(n, values, marker="o", markersize=3.5, color=color, label=f"q={q:g}")
    ax.set_xlabel("packet degree n")
    ax.set_ylabel("raising coefficient")
    ax.set_title(r"q-deformed positive-series ladder at $\kappa=0.8$")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "03_q_ladder_coefficients")
    generated.append("03_q_ladder_coefficients")
```

```
    fig, ax = diagram_axes((12.8, 6.2))
    ax.text(0.5, 0.94, "The symmetric coproduct routes one lowering event through every factor", ha="center", fontsize=20, weight="bold")
    centers = [(0.16, 0.60), (0.50, 0.60), (0.84, 0.60)]
    for index, center in enumerate(centers, start=1):
        box(ax, (center[0] - 0.09, center[1] - 0.08), 0.18, 0.16, f"packet {index}\n$D^+_{{\\kappa_{index},q}}$", face=COLORS[index - 1] + "1C", edge=COLORS[index - 1])
    ax.text(0.5, 0.35, r"$\Delta^{(3)}K_- = K_-q^{K_0}q^{K_0} + q^{-K_0}K_-q^{K_0} + q^{-K_0}q^{-K_0}K_-$", ha="center", fontsize=14)
    for center, color in zip(centers, COLORS[:3]):
        arrow(ax, (center[0], center[1] - 0.08), (0.50, 0.27), color=color)
    box(ax, (0.38, 0.11), 0.24, 0.12, "one exact\nlowest-weight projector", face=PURPLE + "1C", edge=PURPLE)
    ax.text(0.5, 0.04, "The q-powers retain the ordering data needed by the deformed tensor product.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "04_q_coproduct_paths")
    generated.append("04_q_coproduct_paths")

    total = 9
    fig, ax = plt.subplots(figsize=(11.8, 5.5))
    for q, color in zip((0.35, 0.55, 0.75, 0.9, 1.0), COLORS[:5]):
        probabilities = q_two_body_lowest_coefficients(total, 0.7, 1.1, q) ** 2
        ax.plot(range(total + 1), probabilities, marker="o", color=color, label=f"q={q:g}")
    ax.set_xlabel("first-factor excitation n")
    ax.set_ylabel("packet probability")
    ax.set_title("q-Hahn lowest-weight packet redistributes excitation")
    ax.grid(alpha=0.18)
    ax.legend(ncol=3)
    save_figure(fig, "05_q_hahn_packet_distribution")
    generated.append("05_q_hahn_packet_distribution")

    q_values = 1.0 - 10.0 ** (-np.linspace(1.0, 5.0, 45))
    classical = q_two_body_lowest_coefficients(9, 0.7, 1.1, 1.0)
    errors = []
    for value in q_values:
        vector = q_two_body_lowest_coefficients(9, 0.7, 1.1, float(value))
        if np.dot(vector, classical) < 0:
            vector *= -1
        errors.append(np.max(np.abs(vector - classical)))
    fig, ax = plt.subplots(figsize=(11.8, 5.2))
    ax.loglog(1.0 - q_values, errors, marker="o", color=TEAL)
    ax.set_xlabel(r"$1-q$")
    ax.set_ylabel("maximum coefficient error")
    ax.set_title("q-Hahn packets converge to the classical lowest-weight packet")
    ax.grid(alpha=0.18, which="both")
    save_figure(fig, "06_q_hahn_classical_limit")
    generated.append("06_q_hahn_classical_limit")

    q_values = np.linspace(0.02, 1.0, 320)
    cosines = q_values / (1.0 + q_values**2)
    sines = np.sqrt(1.0 + q_values**2 + q_values**4) / (1.0 + q_values**2)
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    ax.plot(q_values, cosines, color=BLUE, linewidth=2.2, label=r"$q/(1+q^2)$")
    ax.plot(q_values, sines, color=ORANGE, linewidth=2.2, label=r"$\sqrt{1+q^2+q^4}/(1+q^2)$")
    ax.set_xlabel("q")
    ax.set_ylabel("recoupling amplitude")
    ax.set_title("Exact equal-shape one-excitation q-beam splitter")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "07_q_beam_splitter_amplitudes")
    generated.append("07_q_beam_splitter_amplitudes")

    fig, axes = plt.subplots(1, 3, figsize=(12.8, 4.2))
    for ax, q in zip(axes, (0.15, 0.55, 1.0)):
        matrix = equal_shape_one_excitation_matrix(q) ** 2
        image = ax.imshow(matrix, cmap="viridis", vmin=0, vmax=1)
        for i in range(2):
            for j in range(2):
                ax.text(j, i, f"{matrix[i,j]:.3f}", ha="center", va="center", color=WHITE if matrix[i,j] > 0.55 else INK)
        ax.set_xticks([0, 1])
```

```
        ax.set_yticks([0, 1])
        ax.set_title(f"q={q:g}")
    fig.suptitle("The q-beam splitter interpolates between routing and mixing", fontsize=16, weight="bold")
    fig.colorbar(image, ax=axes, fraction=0.025, pad=0.025, label="transition probability")
    fig.subplots_adjust(left=0.06, right=0.92, bottom=0.12, top=0.82, wspace=0.25)
    save_figure(fig, "08_q_beam_splitter_kernel")
    generated.append("08_q_beam_splitter_kernel")

    entropy = []
    gap = []
    for q in q_values:
        probability = equal_shape_one_excitation_matrix(float(q)) ** 2
        entropy.append(row_entropies(probability)[0])
        gap.append(spectral_mixing_gap(probability))
    fig, ax = plt.subplots(figsize=(11.8, 5.4))
    ax.plot(q_values, entropy, color=BLUE, linewidth=2.2, label="row entropy")
    ax.plot(q_values, gap, color=ORANGE, linewidth=2.2, label="absolute spectral gap")
    ax.set_xlabel("q")
    ax.set_ylabel("value")
    ax.set_title("One-excitation mixing strengthens toward the classical endpoint")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "09_beam_splitter_entropy_gap")
    generated.append("09_beam_splitter_entropy_gap")

    total = 10
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    for q, color in zip((0.45, 0.65, 0.82, 1.0), (BLUE, TEAL, ORANGE, MAGENTA)):
        spectrum = q_pair_spectrum(total, 0.7, 1.1, q)
        ax.plot(range(total + 1), spectrum, marker="o", markersize=3.5, color=color, label=f"q={q:g}")
    ax.set_xlabel("intermediate packet label")
    ax.set_ylabel("q-Casimir eigenvalue")
    ax.set_title("q-Casimir spectra remain simple and ordered")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "10_q_casimir_spectra")
    generated.append("10_q_casimir_spectra")

    fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.6))
    vmax = 0.0
    matrices = []
    for q in (0.45, 0.72, 1.0):
        matrix = q_three_body_recoupling(10, (0.7, 1.1, 0.9), q).overlap
        matrices.append(matrix)
        vmax = max(vmax, np.max(np.abs(matrix)))
    for ax, matrix, q in zip(axes, matrices, (0.45, 0.72, 1.0)):
        image = ax.imshow(matrix, cmap="coolwarm", vmin=-vmax, vmax=vmax, aspect="auto")
        ax.set_title(f"q={q:g}")
        ax.set_xlabel(r"$C_{23}$ label")
        ax.set_ylabel(r"$C_{12}$ label")
    fig.suptitle("q-Racah packet transforms across the deformation", fontsize=16, weight="bold")
    fig.colorbar(image, ax=axes, fraction=0.022, pad=0.025, label="overlap amplitude")
    fig.subplots_adjust(left=0.06, right=0.92, bottom=0.12, top=0.82, wspace=0.25)
    save_figure(fig, "11_q_racah_heatmaps")
    generated.append("11_q_racah_heatmaps")

    fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.6))
    for ax, matrix, q in zip(axes, matrices, (0.45, 0.72, 1.0)):
        image = ax.imshow(matrix**2, cmap="viridis", vmin=0, vmax=np.max(matrix**2), aspect="auto")
        ax.set_title(f"q={q:g}")
        ax.set_xlabel("measurement output")
        ax.set_ylabel("input packet")
    fig.suptitle("Squared q-Racah amplitudes remain bistochastic", fontsize=16, weight="bold")
    fig.colorbar(image, ax=axes, fraction=0.022, pad=0.025, label="probability")
    fig.subplots_adjust(left=0.06, right=0.92, bottom=0.12, top=0.82, wspace=0.25)
    save_figure(fig, "12_q_racah_probability_kernels")
    generated.append("12_q_racah_probability_kernels")

    totals = np.arange(1, 17)
    fig, ax = plt.subplots(figsize=(11.8, 5.2))
    for q, color in zip((0.45, 0.65, 0.82, 1.0), (BLUE, TEAL, ORANGE, MAGENTA)):
```

```
        residuals = []
        for total in totals:
            matrix = q_three_body_recoupling(int(total), (0.7, 1.1, 0.9), q).overlap
            residuals.append(np.max(np.abs(matrix @ matrix.T - np.eye(total + 1))))
        ax.semilogy(totals, residuals, marker="o", markersize=3.5, color=color, label=f"q={q:g}")
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("maximum orthogonality residual")
    ax.set_title("q-Racah analysis and synthesis remain numerically exact")
    ax.grid(alpha=0.18, which="both")
    ax.legend()
    save_figure(fig, "13_q_orthogonality_residual")
    generated.append("13_q_orthogonality_residual")

    data = q_three_body_recoupling(12, (0.7, 1.1, 0.9), 0.7)
    matrix = data.right_in_left
    fig, ax = plt.subplots(figsize=(8.2, 7.0))
    vmax = np.percentile(np.abs(matrix), 99)
    image = ax.imshow(matrix, cmap="coolwarm", vmin=-vmax, vmax=vmax, aspect="auto")
    ax.set_xlabel(r"$C_{12}$ packet label")
    ax.set_ylabel(r"$C_{12}$ packet label")
    ax.set_title(r"The opposing q-Casimir is tridiagonal in the left atlas")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.03)
    save_figure(fig, "14_opposing_q_casimir")
    generated.append("14_opposing_q_casimir")

    q_poly = 0.73
    N = 8
    A, B, G, D = 0.6, 0.8, -N - 1.0, 0.4
    alpha, beta, gamma, delta = q_poly**A, q_poly**B, q_poly**G, q_poly**D
    values = np.array(
        [[q_racah_polynomial(n, y, alpha, beta, gamma, delta, q_poly).real for y in range(N + 1)] for n in range(N +
  1)]
    )
    fig, ax = plt.subplots(figsize=(8.4, 7.2))
    image = ax.imshow(values, cmap="coolwarm", aspect="auto")
    ax.set_xlabel("lattice coordinate y")
    ax.set_ylabel("polynomial degree n")
    ax.set_title("q-Racah basic-hypergeometric packet table")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.03, label="polynomial value")
    save_figure(fig, "15_q_racah_polynomial_table")
    generated.append("15_q_racah_polynomial_table")

    dual_values = np.array(
        [[q_racah_polynomial(y, n, gamma, delta, alpha, beta, q_poly).real for y in range(N + 1)] for n in range(N +
  1)]
    )
    duality = values - dual_values
    fig, ax = plt.subplots(figsize=(7.8, 6.6))
    image = ax.imshow(np.abs(duality), cmap="magma", aspect="auto")
    ax.set_xlabel("y")
    ax.set_ylabel("n")
    ax.set_title("q-Racah duality residual after parameter exchange")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.03, label="absolute residual")
    save_figure(fig, "16_q_racah_duality")
    generated.append("16_q_racah_duality")

    q_limits = 1.0 - 10.0 ** (-np.linspace(0.55, 4.0, 42))
    errors = []
    for q in q_limits:
        alpha, beta, gamma, delta = q**A, q**B, q**G, q**D
        error = 0.0
        for n in range(4):
            for y in range(N + 1):
                q_value = q_racah_polynomial(n, y, alpha, beta, gamma, delta, float(q)).real
                classical_value = classical_racah_polynomial(n, y, A, B, G, D)
                error = max(error, abs(q_value - classical_value))
        errors.append(error)
    fig, ax = plt.subplots(figsize=(11.8, 5.2))
    ax.loglog(1.0 - q_limits, errors, marker="o", color=TEAL)
    ax.set_xlabel(r"$1-q$")
    ax.set_ylabel("maximum polynomial error")
```

```
    ax.set_title("The explicit q-Racah family converges to Racah")
    ax.grid(alpha=0.18, which="both")
    save_figure(fig, "17_q_racah_to_racah")
    generated.append("17_q_racah_to_racah")

    classical = q_three_body_recoupling(10, (0.7, 1.1, 0.9), 1.0).overlap
    matrix_errors = []
    for q in q_limits:
        matrix = q_three_body_recoupling(10, (0.7, 1.1, 0.9), float(q)).overlap
        matrix = align_signed_matrix(classical, matrix)
        matrix_errors.append(np.max(np.abs(matrix - classical)))
    fig, ax = plt.subplots(figsize=(11.8, 5.2))
    ax.loglog(1.0 - q_limits, matrix_errors, marker="o", color=BLUE)
    ax.set_xlabel(r"$1-q$")
    ax.set_ylabel("maximum matrix error")
    ax.set_title("The packet recoupling matrix has the same classical limit")
    ax.grid(alpha=0.18, which="both")
    save_figure(fig, "18_recoupling_classical_limit")
    generated.append("18_recoupling_classical_limit")

    deformation = deformation_generator(8, (0.7, 1.1, 0.9), theta=0.38)
    generator = np.asarray(deformation["generator"])
    fig, ax = plt.subplots(figsize=(8.2, 7.0))
    vmax = np.max(np.abs(generator))
    image = ax.imshow(generator, cmap="coolwarm", vmin=-vmax, vmax=vmax, aspect="auto")
    ax.set_xlabel("packet coordinate")
    ax.set_ylabel("packet coordinate")
    ax.set_title(r"Skew deformation generator $A=R^{\mathrm{T}}\partial_\theta R$")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.03)
    save_figure(fig, "19_deformation_generator")
    generated.append("19_deformation_generator")

    thetas = np.linspace(0.02, 1.25, 80)
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    for total, color in zip((1, 3, 6, 9), (BLUE, TEAL, ORANGE, MAGENTA)):
        metric = [float(deformation_generator(total, (0.7, 1.1, 0.9), float(theta))["metric"]) for theta in thetas]
        ax.plot(thetas, metric, color=color, linewidth=2.0, label=f"N={total}")
    ax.set_xlabel(r"deformation coordinate $\theta=-\log q$")
    ax.set_ylabel(r"$\frac{1}{2}\|A_N(\theta)\|_F^2$")
    ax.set_title("Packet deformation has a finite transport metric")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "20_deformation_metric")
    generated.append("20_deformation_metric")

    q_grid = np.linspace(0.12, 1.0, 100)
    fig, ax = plt.subplots(figsize=(11.8, 5.4))
    for total, row, color in ((1, 0, BLUE), (2, 1, TEAL), (3, 1, ORANGE), (5, 2, MAGENTA)):
        entropies = []
        for q in q_grid:
            matrix = q_three_body_recoupling(total, (0.5, 0.5, 0.5), float(q)).overlap
            entropies.append(row_entropies(matrix**2)[row])
        ax.plot(q_grid, entropies, color=color, linewidth=2.0, label=f"N={total}, row={row}")
    ax.set_xlabel("q")
    ax.set_ylabel("row entropy")
    ax.set_title("Higher q-Racah channels need not vary monotonically")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "21_entropy_nonmonotonicity")
    generated.append("21_entropy_nonmonotonicity")

    fig, ax = plt.subplots(figsize=(11.8, 5.4))
    for total, color in zip((1, 2, 4, 7), (BLUE, TEAL, ORANGE, MAGENTA)):
        gaps = []
        for q in q_grid:
            probability = q_three_body_recoupling(total, (0.5, 0.5, 0.5), float(q)).overlap**2
            gaps.append(spectral_mixing_gap(probability))
        ax.plot(q_grid, gaps, color=color, linewidth=2.0, label=f"N={total}")
    ax.set_xlabel("q")
    ax.set_ylabel("absolute spectral gap")
    ax.set_title("Measurement mixing depends on both deformation and excitation")
```

```
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "22_stochastic_spectral_gap")
    generated.append("22_stochastic_spectral_gap")

    fig, ax = diagram_axes((12.8, 6.6))
    ax.text(0.5, 0.95, "Five q-coupling trees form the same associahedral pentagon", ha="center", fontsize=20, weight="bold")
    angles = np.linspace(math.pi / 2, math.pi / 2 + 2 * math.pi, 6)[:-1]
    points = [(0.5 + 0.34 * math.cos(angle), 0.50 + 0.34 * math.sin(angle)) for angle in angles]
    labels = [name for name, _ in FOUR_TREES]
    for index, (point, label) in enumerate(zip(points, labels)):
        box(ax, (point[0] - 0.09, point[1] - 0.055), 0.18, 0.11, label, face=COLORS[index] + "1C", edge=COLORS[index], fontsize=9.5)
    for index in range(5):
        arrow(ax, points[index], points[(index + 1) % 5], color=INK)
    ax.text(0.5, 0.50, r"$\prod_{e\in\partial K_4} U_e(q)=I$", ha="center", va="center", fontsize=16, weight="bold")
    ax.text(0.5, 0.08, "Changing q changes every edge transform but not the closed associator path.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "23_q_pentagon_trees")
    generated.append("23_q_pentagon_trees")

    q_grid_pent = np.linspace(0.45, 1.0, 24)
    totals = (1, 2, 3, 4, 5)
    residual_grid = np.zeros((len(totals), len(q_grid_pent)))
    for i, total in enumerate(totals):
        for j, q in enumerate(q_grid_pent):
            trees = q_four_body_tree_bases(total, (0.55, 0.8, 1.05, 1.3), float(q))
            product_matrix, _ = q_pentagon_product(trees)
            residual_grid[i, j] = np.max(np.abs(product_matrix - np.eye(product_matrix.shape[0])))
    fig, ax = plt.subplots(figsize=(11.8, 5.7))
    image = ax.imshow(np.log10(np.maximum(residual_grid, 1e-18)), cmap="viridis", aspect="auto", origin="lower", extent=[q_grid_pent[0], q_grid_pent[-1], totals[0]-0.5, totals[-1]+0.5])
    ax.set_xlabel("q")
    ax.set_ylabel("total excitation N")
    ax.set_title("q-pentagon residual remains at floating-point scale")
    fig.colorbar(image, ax=ax, fraction=0.035, pad=0.025, label=r"$\log_{10}$ residual")
    save_figure(fig, "24_q_pentagon_residual")
    generated.append("24_q_pentagon_residual")

    trees = q_four_body_tree_bases(3, (0.55, 0.8, 1.05, 1.3), 0.68)
    _, overlaps = q_pentagon_product(trees)
    fig, axes = plt.subplots(1, 5, figsize=(14.5, 3.6))
    vmax = max(np.max(np.abs(matrix)) for matrix in overlaps)
    for index, (ax, matrix) in enumerate(zip(axes, overlaps), start=1):
        image = ax.imshow(matrix, cmap="coolwarm", vmin=-vmax, vmax=vmax, aspect="auto")
        ax.set_title(f"edge {index}")
        ax.set_xticks([])
        ax.set_yticks([])
    fig.suptitle("The five signed q-associator edges", fontsize=16, weight="bold")
    fig.colorbar(image, ax=axes, fraction=0.018, pad=0.02)
    fig.subplots_adjust(left=0.04, right=0.94, bottom=0.08, top=0.78, wspace=0.10)
    save_figure(fig, "25_q_pentagon_edge_matrices")
    generated.append("25_q_pentagon_edge_matrices")

    fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.6))
    for ax, q in zip(axes, (0.5, 0.75, 1.0)):
        trees = q_four_body_tree_bases(3, (0.55, 0.8, 1.05, 1.3), q)
        matrix = trees[0].basis.T @ trees[1].basis
        image = ax.imshow(matrix, cmap="coolwarm", vmin=-np.max(np.abs(matrix)), vmax=np.max(np.abs(matrix)), aspect="auto")
        ax.set_title(f"q={q:g}")
        ax.set_xticks([])
        ax.set_yticks([])
    fig.suptitle("One associator edge deforms continuously", fontsize=16, weight="bold")
    fig.colorbar(image, ax=axes, fraction=0.022, pad=0.025)
    fig.subplots_adjust(left=0.05, right=0.92, bottom=0.08, top=0.80, wspace=0.20)
    save_figure(fig, "26_associator_deformation")
    generated.append("26_associator_deformation")

    totals = np.arange(1, 14)
```

```
    rotation_counts = []
    residuals = []
    for total in totals:
        matrix = q_three_body_recoupling(int(total), (0.7, 1.1, 0.9), 0.72).overlap
        rotations, diagonal = givens_decompose(matrix)
        reconstruction = givens_reconstruct(rotations, diagonal)
        rotation_counts.append(len(rotations))
        residuals.append(np.max(np.abs(reconstruction - matrix)))
    fig, ax = plt.subplots(figsize=(11.8, 5.4))
    ax.plot(totals, rotation_counts, marker="o", color=BLUE, label="used rotations")
    ax.plot(totals, totals * (totals + 1) / 2, linestyle="--", color=ORANGE, label=r"$d(d-1)/2$ bound")
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("two-coordinate rotations")
    ax.set_title("Finite q-Racah transforms compile into Givens circuits")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "27_q_givens_counts")
    generated.append("27_q_givens_counts")

    fig, ax = plt.subplots(figsize=(11.8, 5.1))
    ax.semilogy(totals, residuals, marker="o", color=TEAL)
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("maximum reconstruction residual")
    ax.set_title("The compiled q-transform reconstructs the packet matrix")
    ax.grid(alpha=0.18, which="both")
    save_figure(fig, "28_q_givens_residual")
    generated.append("28_q_givens_residual")

    aw_q = 0.7
    aw_parameters = (0.2, 0.3, -0.25, 0.4)
    theta = np.linspace(0.08, math.pi - 0.08, 360)
    fig, ax = plt.subplots(figsize=(11.8, 5.5))
    for n, color in zip(range(5), COLORS[:5]):
        values = [askey_wilson_r(n, np.exp(1j * value), *aw_parameters, aw_q).real for value in theta]
        ax.plot(np.cos(theta), values, color=color, linewidth=1.8, label=f"n={n}")
    ax.set_xlabel(r"$x=\cos\theta$")
    ax.set_ylabel("normalized Askey-Wilson value")
    ax.set_title("The continuous Askey-Wilson packet lift")
    ax.grid(alpha=0.18)
    ax.legend(ncol=3)
    save_figure(fig, "29_askey_wilson_curves")
    generated.append("29_askey_wilson_curves")

    residual_table = np.zeros((6, 36))
    theta_grid = np.linspace(0.12, math.pi - 0.12, residual_table.shape[1])
    for n in range(residual_table.shape[0]):
        function = lambda value, degree=n: askey_wilson_r(degree, value, *aw_parameters, aw_q)
        eigenvalue = aw_q ** (-n) + np.prod(aw_parameters) * aw_q ** (n - 1)
        for j, value in enumerate(theta_grid):
            z = np.exp(1j * value)
            residual_table[n, j] = abs(askey_wilson_operator(function, z, *aw_parameters, aw_q) - eigenvalue *
     function(z))
    fig, ax = plt.subplots(figsize=(11.8, 5.4))
    image = ax.imshow(np.log10(np.maximum(residual_table, 1e-18)), cmap="magma", aspect="auto", origin="lower",
     extent=[theta_grid[0], theta_grid[-1], -0.5, 5.5])
    ax.set_xlabel(r"$\theta$")
    ax.set_ylabel("degree n")
    ax.set_title("Askey-Wilson q-difference equation residual")
    fig.colorbar(image, ax=ax, fraction=0.035, pad=0.025, label=r"$\log_{10}$ residual")
    save_figure(fig, "30_askey_wilson_difference_residual")
    generated.append("30_askey_wilson_difference_residual")

    fig, ax = diagram_axes((12.8, 6.4))
    ax.text(0.5, 0.94, "The finite and continuous q-functions belong to one transfer class", ha="center", fontsize
     =20, weight="bold")
    box(ax, (0.06, 0.58), 0.20, 0.16, "Uq(su(1,1))\nq-Hahn packets", face=TEAL + "1C", edge=TEAL)
    box(ax, (0.40, 0.58), 0.20, 0.16, "finite q-Racah\nrecoupling", face=MAGENTA + "1C", edge=MAGENTA)
    box(ax, (0.74, 0.58), 0.20, 0.16, "continuous\nAskey-Wilson lift", face=BLUE + "1C", edge=BLUE)
    arrow(ax, (0.26, 0.66), (0.40, 0.66), text="pair Casimir")
    arrow(ax, (0.60, 0.66), (0.74, 0.66), text="continuous spectrum")
    box(ax, (0.18, 0.20), 0.22, 0.15, "q -> 1\nclassical Racah", face=ORANGE + "1C", edge=ORANGE)
```

```
    box(ax, (0.60, 0.20), 0.22, 0.15, "q -> 0\nsparse routing", face=YELLOW + "25", edge=YELLOW)
    arrow(ax, (0.50, 0.58), (0.29, 0.35), color=ORANGE)
    arrow(ax, (0.50, 0.58), (0.71, 0.35), color=YELLOW)
    ax.text(0.5, 0.07, "The packet calculus keeps the exact reconstruction law while q changes the local transfer geometry.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "31_q_function_class_map")
    generated.append("31_q_function_class_map")

    fig, ax = diagram_axes((13.0, 6.4))
    ax.text(0.5, 0.94, "A programmable q-packet execution path", ha="center", fontsize=20, weight="bold")
    stages = [
        (0.04, 0.58, 0.16, 0.16, "packet shapes\n$\\kappa_i$", BLUE),
        (0.24, 0.58, 0.16, 0.16, "choose q and\nexcitation N", TEAL),
        (0.44, 0.58, 0.16, 0.16, "build q-Casimir\nprojectors", ORANGE),
        (0.64, 0.58, 0.16, 0.16, "diagonalize\nmultiplicity", MAGENTA),
        (0.84, 0.58, 0.13, 0.16, "compile\nrotations", RED),
    ]
    for x, y, w, h, label, color in stages:
        box(ax, (x, y), w, h, label, face=color + "1C", edge=color, fontsize=11)
    for left, right in zip(stages[:-1], stages[1:]):
        arrow(ax, (left[0] + left[2], left[1] + left[3] / 2), (right[0], right[1] + right[3] / 2))
    box(ax, (0.16, 0.20), 0.20, 0.15, "signed\nq-transform", face=PURPLE + "1C", edge=PURPLE)
    box(ax, (0.40, 0.20), 0.20, 0.15, "measurement\nkernel", face=YELLOW + "25", edge=YELLOW)
    box(ax, (0.64, 0.20), 0.20, 0.15, "pentagon\ncertificate", face=TEAL + "1C", edge=TEAL)
    arrow(ax, (0.90, 0.58), (0.26, 0.35), color=PURPLE)
    arrow(ax, (0.90, 0.58), (0.50, 0.35), color=YELLOW)
    arrow(ax, (0.90, 0.58), (0.74, 0.35), color=TEAL)
    ax.text(0.5, 0.07, "Dense tensor spaces are used only to define the projector. Execution takes place in the smaller multiplicity packet.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "32_q_packet_compiler")
    generated.append("32_q_packet_compiler")

    return generated


def write_tables(checks: dict[str, dict[str, float | int | str | bool]]) -> None:
    with (TABLES / "check_summary.csv").open("w", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(["check", "passed", "detail"])
        for name, result in checks.items():
            detail = ", ".join(f"{key}={value}" for key, value in result.items() if key != "passed")
            writer.writerow([name, result["passed"], detail])

    with (TABLES / "q_beam_splitter.csv").open("w", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(["q", "cosine", "sine", "row_entropy", "spectral_gap"])
        for q in (0.1, 0.25, 0.5, 0.75, 1.0):
            matrix = equal_shape_one_excitation_matrix(q)
            probability = matrix**2
            writer.writerow([
                q,
                matrix[0, 0],
                matrix[0, 1],
                row_entropies(probability)[0],
                spectral_mixing_gap(probability),
            ])

    with (TABLES / "q_pentagon.csv").open("w", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(["q", "N", "dimension", "pentagon_residual"])
        for q in (0.5, 0.7, 0.85, 1.0):
            for total in (1, 2, 3, 4):
                trees = q_four_body_tree_bases(total, (0.55, 0.8, 1.05, 1.3), q)
                product_matrix, _ = q_pentagon_product(trees)
                writer.writerow([q, total, product_matrix.shape[0], np.max(np.abs(product_matrix - np.eye(product_matrix.shape[0])))])


def write_manifest(generated: Sequence[str], checks: dict[str, dict[str, float | int | str | bool]]) -> None:
    manifest = {
        "title": "Projector-Generated Special Functions VIII",
```

```
        "author": "Komorebi P.",
        "figures": list(generated),
        "figure_pairs": len(generated),
        "checks_passed": sum(1 for value in checks.values() if bool(value["passed"])),
        "checks_total": len(checks),
    }
    (RESULTS / "manifest.json").write_text(json.dumps(jsonable(manifest), indent=2))


def main() -> None:
    checks = run_checks()
    generated = generate_figures()
    write_tables(checks)
    write_manifest(generated, checks)
    passed = sum(1 for result in checks.values() if bool(result["passed"]))
    print(f"Generated {len(generated)} figure pairs")
    print(f"Checks passed: {passed}/{len(checks)}")
    if passed != len(checks):
        raise SystemExit(1)


if __name__ == "__main__":
    main()
```

Chapter C

Bibliographic note

Bibliography

[1] NIST Digital Library of Mathematical Functions, Section 18.28, "Askey-Wilson Class."

[2] H. T. Koelink and J. Van der Jeugt, "Convolutions for orthogonal polynomials from Lie and quantum algebra representations," SIAM Journal on Mathematical Analysis 29 (1998), 794–822.

[3] E. Koelink and H. Rosengren, "Harmonic analysis on the $SU(2)$ dynamical quantum group," Acta Applicandae Mathematicae 69 (2001), 163–220.

[4] Y. Shibukawa, "Clebsch–Gordan Coefficients for $\mathcal{U}_q(\mathfrak{su}(1,1))$ and $\mathcal{U}_q(\mathfrak{sl}(2))$, and Linearization Formula of Matrix Elements," Publ. Res. Inst. Math. Sci. 28 (1992), 775–807.

[5] V. X. Genest, P. Iliev, and L. Vinet, "Coupling coefficients of $su_q(1,1)$ and multivariate q-Racah polynomials," Nuclear Physics B 927 (2018), 97–123.

[6] G. Gasper and M. Rahman, *Basic Hypergeometric Series*, second edition, Cambridge University Press, 2004.

[7] S. Wei, Y. Zhai, Q. Lu, et al., "The Riemann Hypothesis manifested in dynamical quantum phase transitions," Nature Communications, accepted 18 June 2026, DOI 10.1038/s41467-026-74935-8.

Chapter D

Reproduction

From the source directory, run

```
python projector_generated_special_functions_viii.py
python -m pytest -q
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VIII.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VIII.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VIII.tex
```

The Python step writes all 32 figure pairs, the check ledger, the selected data tables, and the machine-readable manifest before the paper is compiled.